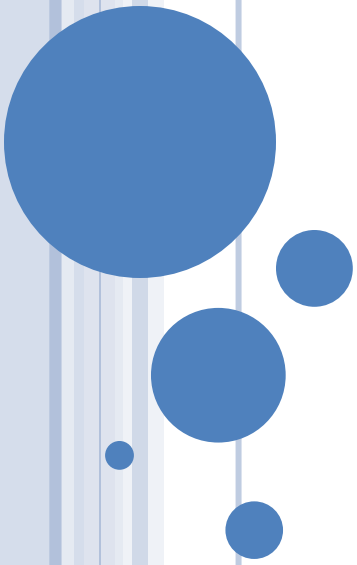


Теория автоматов и формальных языков

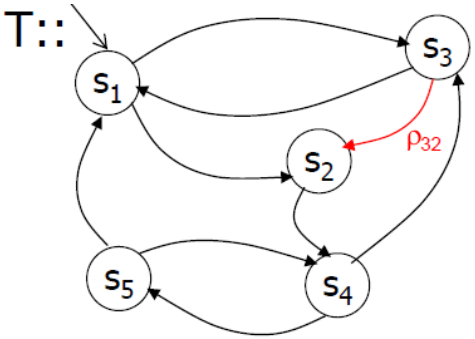
Применения и расширения конечных автоматов

Цепи Маркова



Марковские цепи vs вероятностный автомат

- Автомат T может находиться в одном из N состояний, $S=\{s_1, \ldots, s_N\}$ и может изменять свое состояние случайно в дискретные моменты времени
- Вероятность ρ_{ij} того, что из состояния s_i система перейдет в состояние s_j, зависит только от состояния s_i а не от истории, не от того, как она пришла в s_i
- Вероятности переходов задаются матрицей P переходных вероятностей



состояния

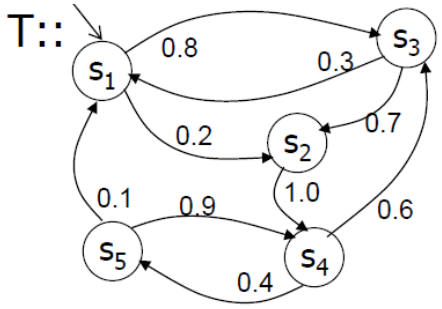
состояния	P	s_1	s_2	s_3	s_4	s_5
	s_1	ρ_{11}	ρ_{12}	ρ_{13}	ρ_{14}	ρ_{15}
	s_2	ρ_{21}	ρ_{22}	ρ_{23}	ρ_{24}	ρ_{25}
	s_3	ρ_{31}	ρ_{32}	ρ_{33}	ρ_{34}	ρ_{35}
	s_4	ρ_{41}	ρ_{42}	ρ_{43}	ρ_{44}	ρ_{45}
	s_5	ρ_{51}	ρ_{52}	ρ_{53}	ρ_{54}	ρ_{55}

Если перехода из s_i в s_k нет, то $\rho_{ik}=0$

Стохастические матрицы

- Однородные цепи Маркова – матрица P не зависит от времени (вероятности ρ_{ij} – постоянны)
- T – цепь Маркова, если:
  - $0 \geq \rho_{ij} \geq 1$
  - $\Sigma_{1 \geq j \geq N} \rho_{ij} = 1$ сумма вероятностей перехода ИЗ любого состояния $s_i = 1$

Цепь Маркова – череда событий, в которой каждое последующее событие зависит от предыдущего.



В состояния

ИЗ состояния P	s_1	s_2	s_3	s_4	s_5
s_1	0	0.2	0.8	0	0
s_2	0	0	0	1	0
s_3	0.3	0.7	0	0	0
s_4	0	0	0.6	0	0.4
s_5	0.1	0	0.9	0	0

Цепь Маркова

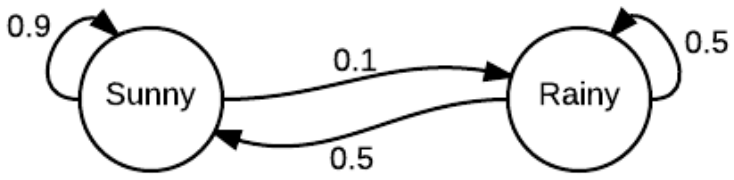
- череда событий, в которой каждое последующее событие зависит от предыдущего.
- распространенный и довольно простой способ моделирования случайных событий. Используется в самых разных областях, начиная генерацией текста и заканчивая финансовым моделированием
- понятна и проста в использовании, т. к. она может быть реализована без использования каких-либо статистических или математических концепций. Цепь Маркова идеально подходит для изучения вероятностного моделирования и Data Science.



Пример

Существует только ***два погодных условия***: может быть либо солнечно, либо пасмурно. Всегда можно безошибочно определить погоду в текущий момент. **Гарантированно** будет ясно или облачно.

Теперь вам захотелось научиться предсказывать погоду на завтрашний день. Интуитивно вы понимаете, что ***погода не может кардинально поменяться за один день***. На это влияет множество факторов. Завтрашняя погода напрямую зависит от текущей и т. д. Таким образом, для того чтобы предсказывать погоду, вы на протяжении нескольких лет собираете данные и приходите к выводу, что после пасмурного дня вероятность солнечного равна 0,25. Логично предположить, что вероятность двух пасмурных дней подряд равна 0,75, так как мы имеем всего два возможных погодных условия.



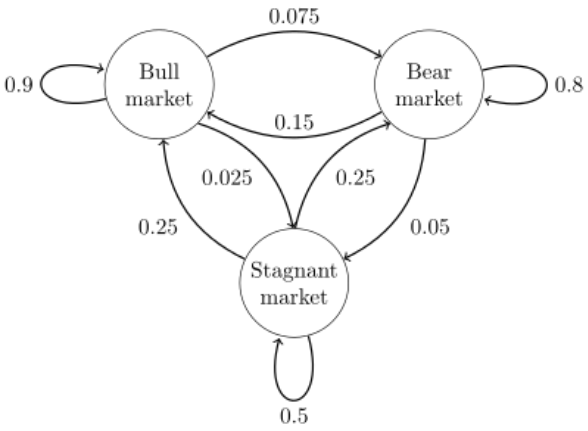
Цепь Маркова состоит из набора переходов, которые определяются распределением вероятностей, которые в свою очередь удовлетворяют **Марковскому свойству**.

Цепь Маркова

- Цепь Маркова состоит из набора переходов, которые определяются распределением вероятностей, которые в свою очередь удовлетворяют Марковскому свойству.
- Распределение вероятностей зависит только от переходов с текущего дня на следующий. Это уникальное свойство ***Марковского процесса*** – он делает это **без использования памяти**.
- Как правило, такой подход не способен создать последовательность, в которой бы наблюдалась какая-либо тенденция. Например, в то время как цепь Маркова способна сымитировать стиль письма, основанный на частоте использования какого-то слова, она **не способна создать тексты с глубоким смыслом**, так как она может работать только с большими текстами. Именно поэтому цепь Маркова не может производить контент, зависящий от контекста.

Формально: цепь Маркова – вероятностный автомат

- Если цепь Маркова имеет N возможных состояний, то матрица будет иметь вид N x N, в которой запись (I, J) будет являться вероятностью перехода из состояния I в состояние J. Кроме того, такая матрица должна быть стохастической, то есть строки или столбцы в сумме должны давать единицу.
- Цепь Маркова имеет начальный вектор состояния, представленный в виде вектора. Он описывает распределения вероятностей начала в каждом из N возможных состояний. Запись I описывает вероятность начала цепи в состоянии I.
- Матрицу перехода P можно определить вычислением (I, J) с помощью возведения P в степень M. Для малых значений M это можно делать вручную, с помощью повторного умножения.
- Для больших значений M более эффективным способом возведения матрицы в степень будет сначала диагонализировать эту матрицу.



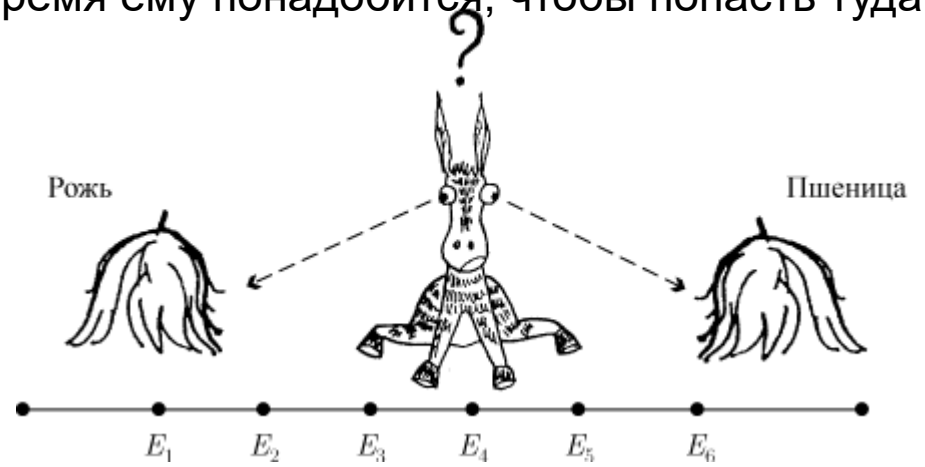
$$P = \begin{bmatrix} 0.9 & 0.075 & 0.025 \\ 0.15 & 0.8 & 0.05 \\ 0.25 & 0.25 & 0.5 \end{bmatrix}.$$

$$x^{(0)} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

Задача об осле

Рассмотрим задачу об осле, стоящем точно между двумя копнами: соломы ржи и соломы пшеницы.

- Осел стоит между двумя копнами: "Рожь" и "Пшеница". Каждую минуту он либо передвигается на десять метров в сторону первой копны (*с вероятностью 1/2*), либо в сторону второй копны (*с вероятностью 1/3*), либо остается там, где стоял (*с вероятностью 1/6*). Такое поведение называется ***одномерным случайным блужданием***.
- Будем предполагать, что обе копны являются *"поглощающими"* в том смысле, что если осел подойдет к одной из копен, то он там и останется. Зная расстояние между двумя копнами и начальное положение осла, можно поставить несколько вопросов, например: у какой копны он очутится с большей вероятностью и какое наиболее вероятное время ему понадобится, чтобы попасть туда?



Задача об осле

Предположим, что расстояние между копнами равно пятидесяти метрам и что наш осел находится в двадцати метрах от копны "Пшеницы".

начальное положение можно задать вектором $x = (0, 0, 0, 1, 0, 0),$

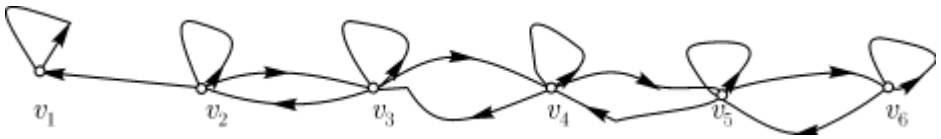
По прошествии одной минуты вероятности его местоположения описываются вектором $(0, 0, 1/2, 1/6, 1/3, 0)$

еще через две минуты — вектором $(0, 1/4, 1/6, 13/36, 1/9, 1/9)$

матрицу перехода

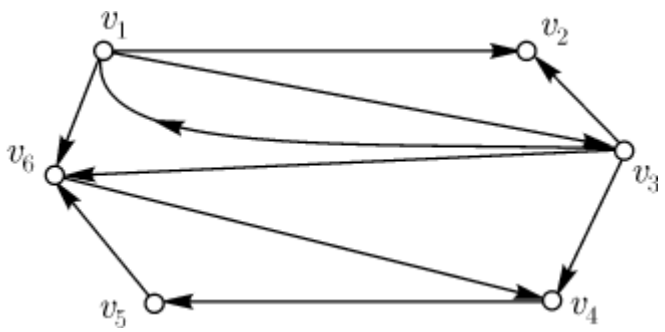
$$\begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 1/2 & 1/6 & 1/3 & 0 & 0 & 0 \\ 0 & 1/2 & 1/6 & 1/3 & 0 & 0 \\ 0 & 0 & 1/2 & 1/6 & 1/3 & 0 \\ 0 & 0 & 0 & 1/2 & 1/6 & 1/3 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix}$$

Из E_4 в E_1 можно попасть за три минуты и вообще нельзя попасть из E_1 в E_4.



Определения

$$\begin{pmatrix} 0 & 1/4 & 1/2 & 0 & 0 & 1/4 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 1/2 & 1/3 & 0 & 1/12 & 0 & 1/12 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{pmatrix}$$



- Цепь Маркова, в которой из любого состояния можно попасть в любое другое, называется *неприводимой*. Ясно, что цепь Маркова неприводима тогда и только тогда, если ее ассоциированный орграф сильно связан.
- Если начальное состояние есть E_i и вероятность возвращения в E_i на некотором более позднем шаге равна единице, то E_i называется *возвратным* (или рекурсивным) состоянием.
- Состояние цепи Маркова называется периодическим с периодом t, если в него можно вернуться только по истечении времени, кратного t.
- Состояние цепи Маркова называют *эргодическим*, если оно одновременно возвратно и непериодично. Если любое состояние цепи Маркова является эргодическим, то назовем ее эргодической цепью.

Непрерывные цепи Маркова

Для непрерывных цепей Маркова вектор вероятностей состояния есть функция времени и определяется путем решения системы дифференциальных уравнений, которые составляются по графу состояния цепи Маркова по следующим правилам :

- в левой части каждого уравнения стоят производные по времени вероятностей состояния цепи;
- правая часть этих уравнений содержит столько членов, сколько переходов (стрелок на графе) связанно с данным состоянием;
- каждый член правой части уравнений равен произведению плотности вероятностей перехода , соответствующей данной стрелки графа состояния, умноженной на вероятность того состояния из которого исходит стрелка;
- каждый член правой части уравнений имеет знак «минус», если стрелка графа состояния входит в данное состояния и знак «плюс», если стрелка выходит из данного состояния.

$$\frac{dp_1(t)}{dt} = -\lambda_{12} p_1(t) + \lambda_{31} p_3(t);$$

$$\frac{dp_2(t)}{dt} = \lambda_{12} p_1(t) - \lambda_{23} p_2(t);$$

$$\frac{dp_3(t)}{dt} = \lambda_{23} p_2(t) - \lambda_{31} p_3(t).$$

$$p_1(t) + p_2(t) + p_3(t) = 1$$

$$p_1(t),\ p_2(t),\ p_3(t)$$

В случае, когда нас интересую вероятности состояния непрерывных цепей Маркова по истечению длительного промежутка времени (установившейся процесс $t \to \infty$), то решение системы получают путем записи в левой части системы дифференциальных вместо производных нулей, т.е. :

$$\frac{dp_n(t)}{dt} = 0.$$

Поток событий

Переход из состояния в состояние в непрерывных цепях Маркова происходит вод воздействием *потока событий.*

Потоком событий называется последовательность однородных событий, следующих одно за другим в какие-то случайные моменты времени.

- Поток событий называю простейшим или стационарным Пуассоновским, если он стационарен, ординарен и без последействия.
- Поток называется стационарным, если вероятность попадания события на участок времени зависит только от длины этого участка и не зависит от места расположения этого участка на оси времени.
- Поток называют потоком без последействия, если для любых непересекающихся участков времени число событий, попадающих на один из них, не зависит от числа событий на другом участке.
- Поток событий называют ординарным, если вероятность попадания на элементарный участок двух и более событий пренебрежительно мала по сравнению с вероятностью попадания на этот участок одного события.

$P_m(\tau) = \frac{(\lambda \cdot \tau)^m}{m!} \cdot e^{-\lambda \tau}$ λ - интенсивность потока событий (для стационарного потока не зависит от времени)

подчиняется **закону распределения Пуассона** с математическим ожиданием $m_x = \lambda \cdot \tau$

Промежутки времени между событиями в Пуассоновском потоке событий подчиняются показательному закону распределения с функцией распределения и плотностью распределения :

$F(x) = 1 - e^{-\lambda t}; \quad \rho(x) = \lambda \cdot e^{-\lambda t}$ $m_x = \frac{1}{\lambda}$ $D_x = \frac{1}{\lambda^2}$ $\sigma_x = \frac{1}{\lambda}$

Распределение Пуассона

- Предельный закон распределения для биномиального при n→∞, так что np = m – постоянно.
- Если объем испытаний не менее нескольких десятков, а вероятность наступления случайного события A в каждом из испытаний мала (p<<1), причем математическое ожидание данного события np не превышает 10.
- *Число вызовов на телефонной сети за время t, число опечаток в большом тексте, число бракованных деталей в большой партии…*

формула Пуассона – «закона редких событий»:

$$P(x = m) = \frac{\mu^m}{m!} e^{-\mu}.$$

$$MX = DX = a,$$

Пример 2.8. Вероятность попадания в цель при одном выстреле равна 0,01. Какова вероятность того, что число попаданий при 200 выстрелах составит не менее 5 и не более 10?

$$P\{5 \leqslant X \leqslant 10\} = P\{X = 5\} + P\{X = 6\} + \ldots + P\{X = 10\}$$

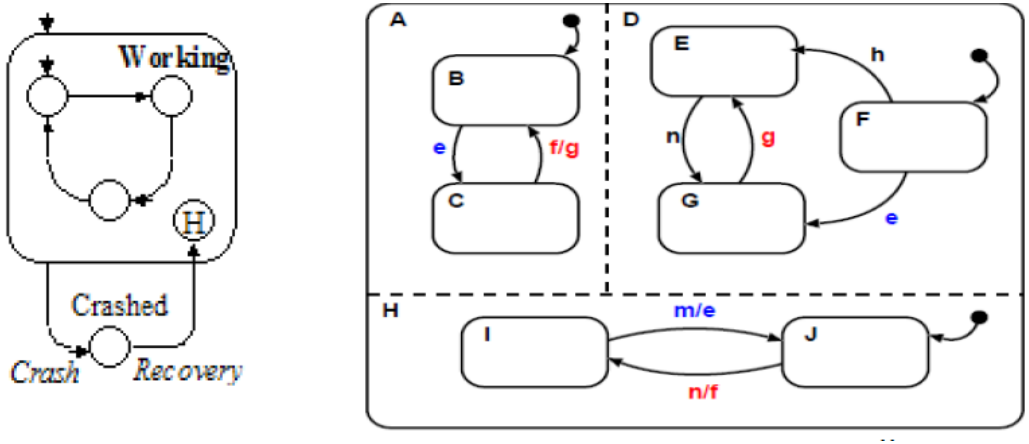
$$a = np = 200 \cdot 0{,}01 = 2,\ e^{-a} = e^{-2} \approx 0{,}135$$

$$P\{5 \leqslant X \leqslant 10\} = 0{,}135\left(\frac{2^5}{5!} + \frac{2^6}{6!} + \frac{2^7}{7!} + \frac{2^8}{8!} + \frac{2^9}{9!} + \frac{2^{10}}{10!}\right) \approx 0{,}053$$

Стейтчарты (Statecharts) – карты состояний



Стейтчарты (Statecharts) – обобщение КА

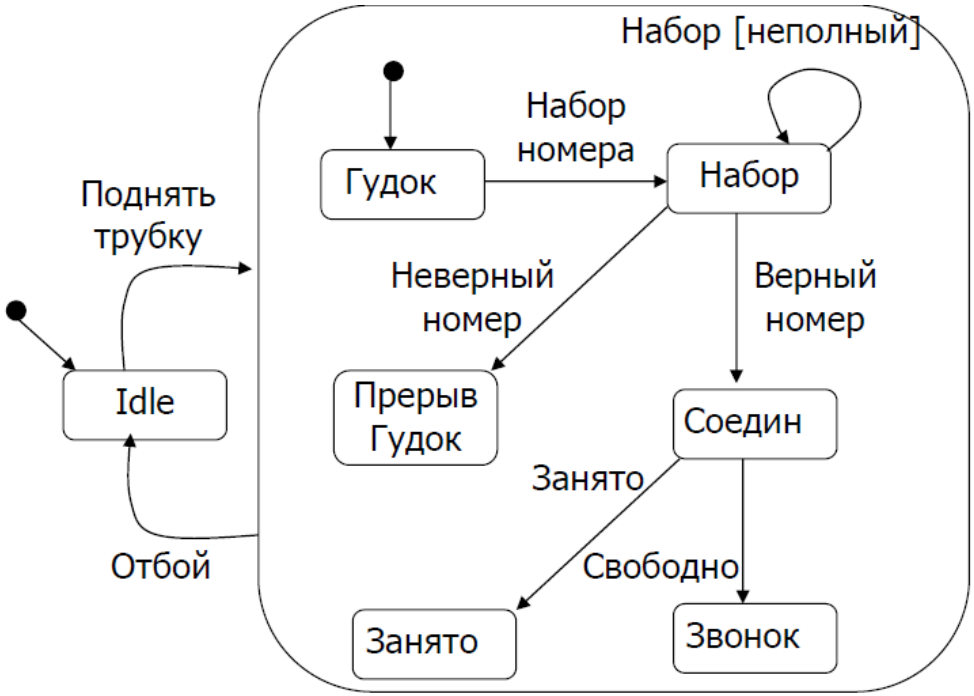


- В стейтчартах кроме визуальных графических состояний есть:
  - иерархия состояний
  - "исторический" переход H
  - параллельное функционирование нескольких процессов
  - переменные, логические выражения и функции от локальных переменных
  - при входе в состояние может выполняться входная функция над локальными переменными, при выходе из состояния может выполняться выходная функция
  - каждый переход определяется
    - внешним событием, либо
    - выполнением логического условия, ставшего истинным при выполнении ДРУГОГО процесса, параллельного данному, либо
    - срабатыванием таймера

Стейтчарты (Statecharts) – карты состояний

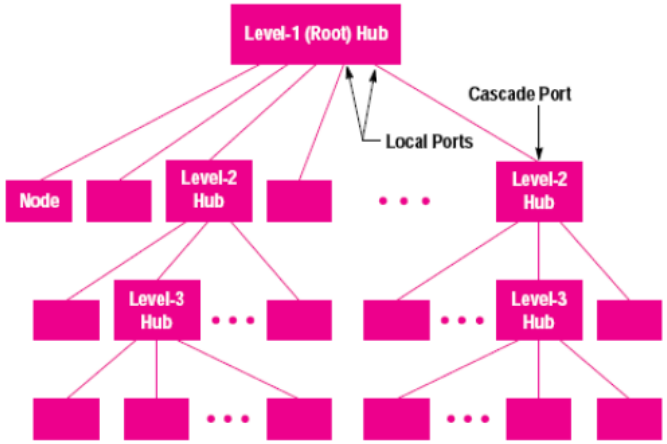
- Системы переходов (фактически, форма конечных автоматов) стали использоваться для графического описания алгоритмов поведения реагирующих систем (reactive systems) с 1968 года
- Многократно этот метод спецификации алгоритмов под разными именами снова "открывался", в частности, "автоматное программирование" (через 25 лет) – в 90-х
- Стейтчарты – *карты состояний* – это визуальный формализм для графического представления поведения сложных систем. Это расширение визуального изображения конечных автоматов для более удобного и компактного задания их поведения
- Стейтчарты были введены в 1987 г. Дэвидом Харелом (David Harel). В работе, опубликованной в журнале Science of Computer Programming, он писал: "*Для реагирующих систем (reactive systems), чье поведение – это реакции на внешние и внутренние события (ОС, драйверы, встроенные системы управления, ...) общепринято, что состояния и переходы – это естественный язык для описания их поведения*". Но сложные системы описать таким образом нельзя – очень громоздко вырисовывать все состояния и все реакции на события в этих состояниях!

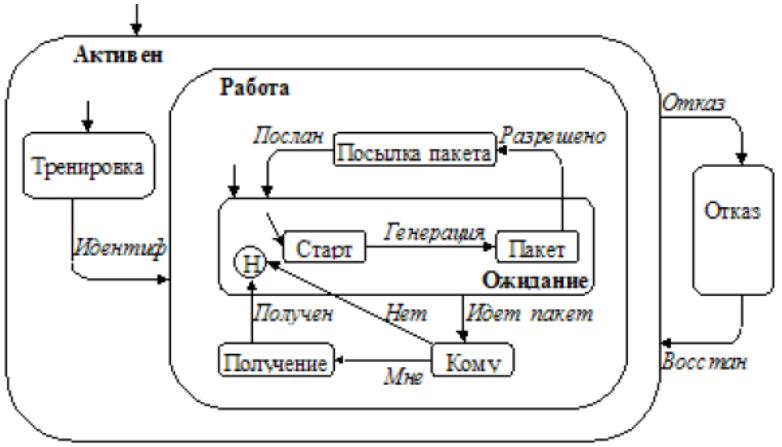
Пример: система управления телефоном



С того момента, как графический язык стейтчартов был предложен в 1987 г., он стал де-факто стандартом для представления алгоритмов управляющего ПО. В частности, по стейтчартам можно автоматически генерировать код управляющей программы. Такие программы были разработаны в конце 80-х

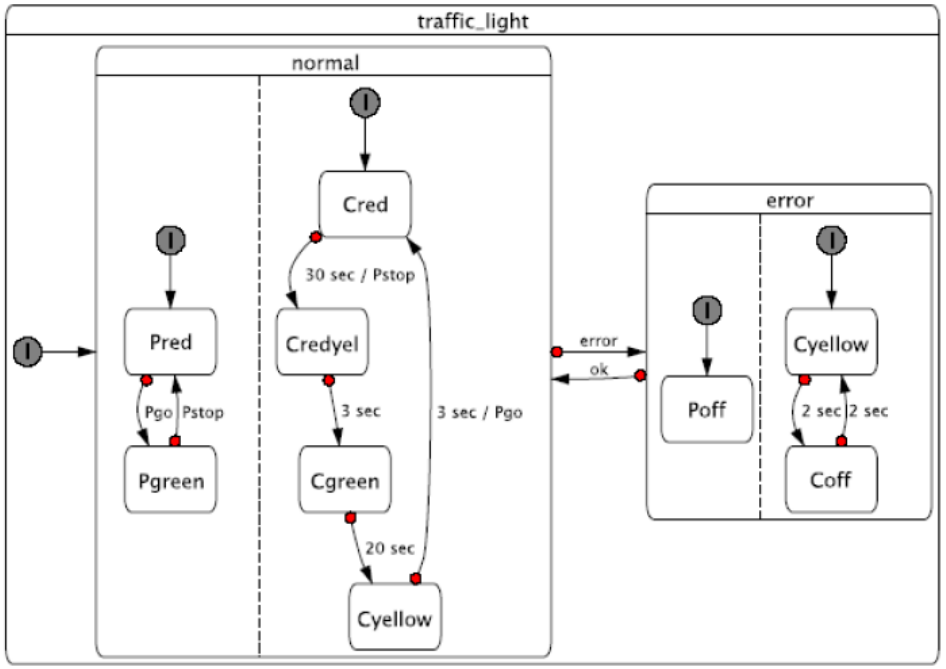
Пример: спецификация протокола IEEE 802.12





- Разработка IBM и HP - альтернатива Fast Ethernet, передача со скоростью 100 Мб/с. Определен новый метод доступа Demand Priority, с выделением обычных и приоритетных сообщений. Приоритетным гарантировано ограниченное время доставки
- Работа начинается с “Тренировки”, в которой определяется архитектура сети. Затем – в состояние “Работа” и “Ожидание”. После генерации сообщения, переходим в “Пакет” – сообщение готово. Если получено разрешение, пакет посылается.
- В любом состоянии гиперсостояния “Ожидание” по сигналу “Идет пакет” прерываемся и начинаем принимать. После приема инф. кадра определяем: мне или нет, и если нет, то возвращаемся из прерывания

Графический язык Esterel для проектирования встроенных систем управления в авиастроении



- Система управления пешеходным переходом (из статьи [1]) в графическом языке Eserel

Параллельно функционируют процессы регулирования траффика и регулирования пешеходным переходом

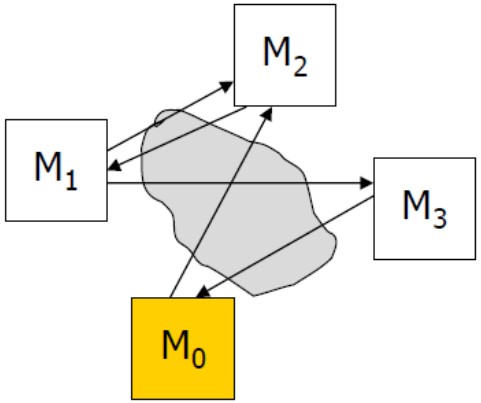
Pred - Pedestrian Red
Pgreen - Ped Green
Poff – Pedestrian Off

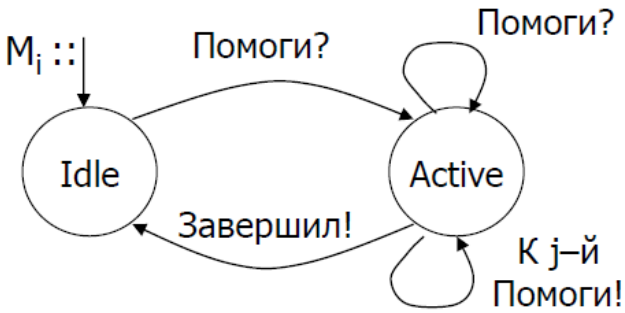
- Для Airbus (крупнейшая авиастроительная компания, Тулуза, 50 тыс. чел) разработка бортовых систем ведется с помощью графического языка Esterel (с 1983) и автоматической генерации кода

1. S. Prochnoff. Comfortable Modeling of Complex Reactive Systems, Uni. Kiel, 2002

Протокол распределенного завершения

Классическая проблема распределенных вычислений





Несколько машин в сети занимаются выполнением подзадач одной задачи. Любая из машин может обратиться за помощью к любой другой машине - послать ей через сеть порцию задачи с необходимыми данными. Решение завершается, когда ни в одной из машин уже ничего не решается

Завершив свои вычисления и отослав результаты, машина становится пассивной. Она может вновь стать активной, только получив запрос о помощи от какой-нибудь другой машины. Проблема состоит в том, чтобы определить, когда в распределенной системе все процессы станут пассивными

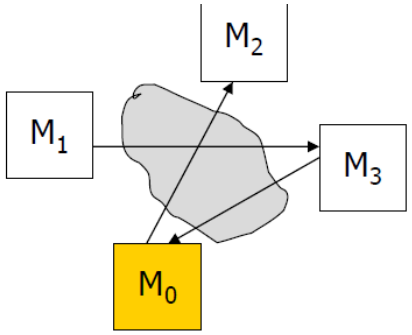
Алгоритм не может состоять просто в проверке всех машин: завершились ли они. Мгновенно этого сделать нельзя. Разработка такого алгоритма является классической проблемой распределенных вычислений

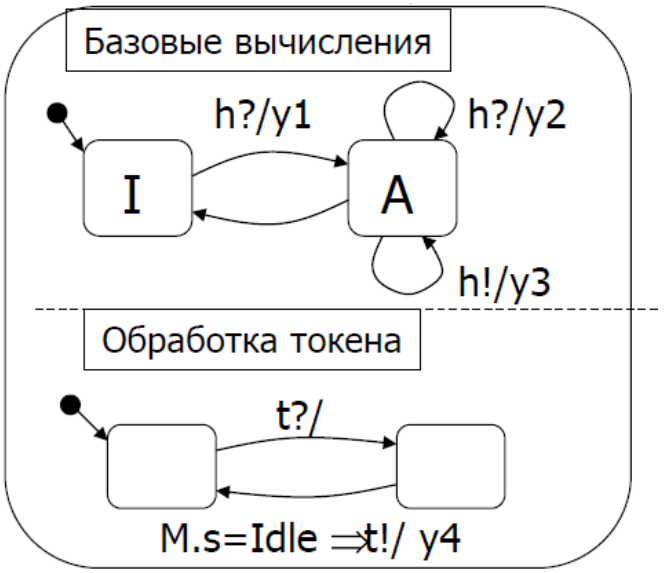
Протокол распределенного завершения Дейкстры

- Алгоритм Дейкстры - повторение циклов зондирования сети посылкой зонда (токена) по виртуальному кольцу машин. Он состоит из следующих локальных правил, которые гарантируют решение глобальной проблемы:
- *Каждая машина в сети поддерживает счетчик с. Каждая посылка базового сообщения (с просьбой о помощи) увеличивает с на 1; прием такого сообщения уменьшает с на 1. Таким образом, сумма всех счетчиков в сети машин в любой момент времени равна числу сообщений, задержанных в сети*
- Машина 0 инициирует шаг зондирования в любой момент и в любом своем состоянии, посылая токен со значением 0 машине N-1. Каждая машина i задерживает полученный токен до тех пор, пока она не станет пассивной. После этого значение токена увеличивается на с, и токен передается машине i-1
- *И машины, и токен имеют цвет. Вначале все машины и токен белые. Машина, получившая базовое сообщение, становится черной. Когда машина передает токен, она становится белой. Если черная машина передает токен, токен становится черным, в противном случае токен сохраняет свой цвет*
- Когда машина 0 получает свой токен, прошедший по кольцу всех машин, очередной цикл зондирования закончен. Машина 0, получив токен, ждет, пока она сама не перейдет в пассивное состояние. После этого она делает заключение о завершении базовых вычислений в сети машин при следующих условиях:
- сама машина 0 белая, И
- полученный токен белый, И
- сумма значений токена и счетчика с машины 0 равна 0.
- *В противном случае завершения на этом цикле не обнаружено, машина 0 запускает новый цикл зондирования*



Алгоритм распределенного завершения: стейтчарт для i-й машины в сети





y1: ci--; Mi.цвет:=черный;

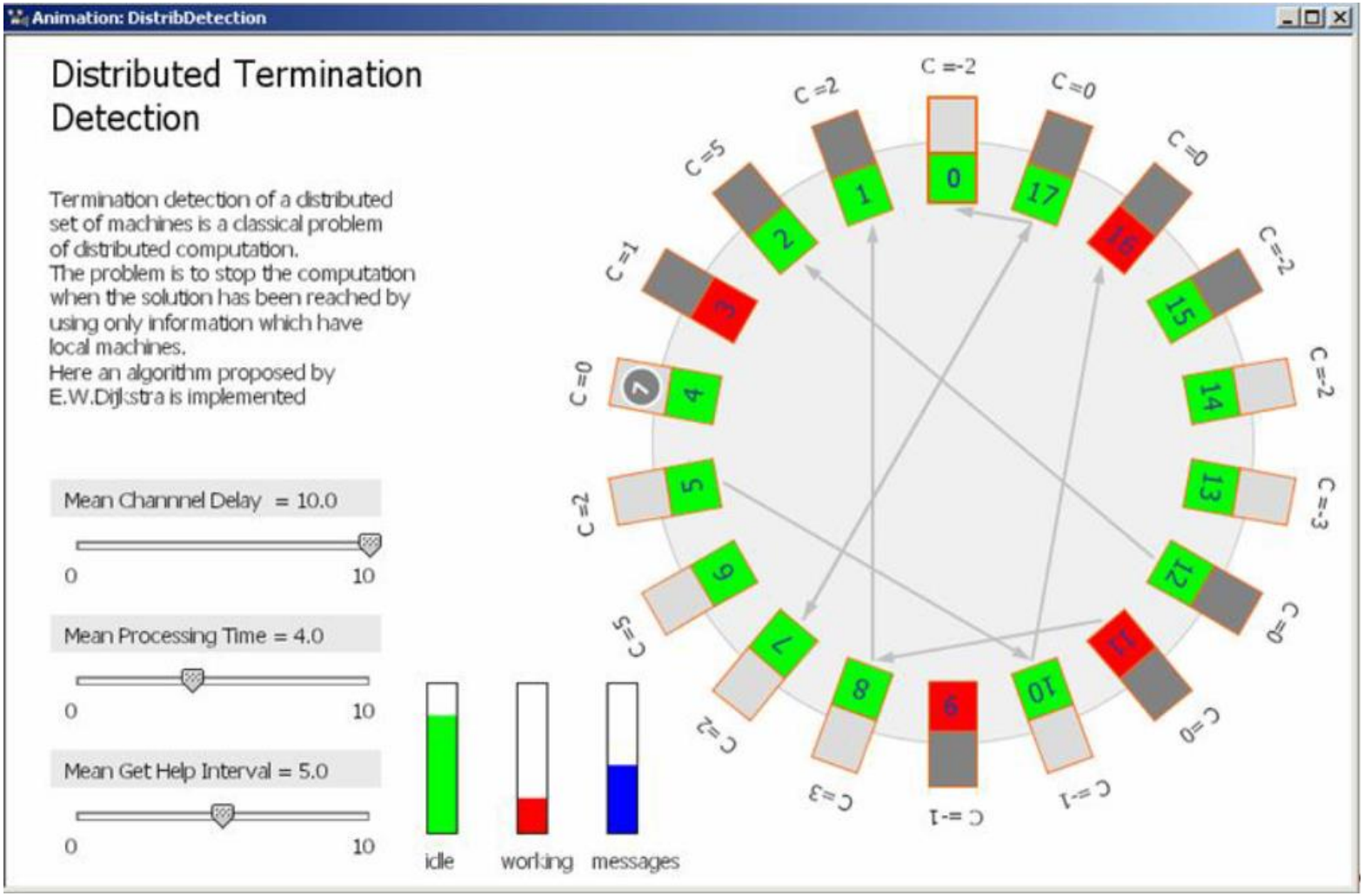
y2: ci--;

y3: ci++;

y4: if Mi.цвет = черный ⇒t.цвет:=черный;
Mi.цвет:=белый; t.c := c + ci;

Стейтчарт для выделенной машины M_0, запускающей токен, чуть сложнее

Работа модели протокола



UML и стейтчарты

Unified Modeling Language - язык графического описания для проектирования ОО ПО. Использует визуальный формализм для создания абстрактной модели системы. UML используют также для моделирования бизнес-процессов, отображения организационных структур и пр.

Содержит диаграммы трех типов - для моделирования статической структуры, поведенческих аспектов и подробностей реализации приложения.

Существует 13 официальных диаграмм UML 2.0, каждая представляет один из аспектов разрабатываемой системы:

- Диаграмма активности;
- Диаграмма классов;
- Диаграмма связей;
- ...
- Диаграмма систем переходов (state machines) – использует стейтчарты
- Диаграмма синхронизации;
- Диаграмма прецедентов (Use cases)

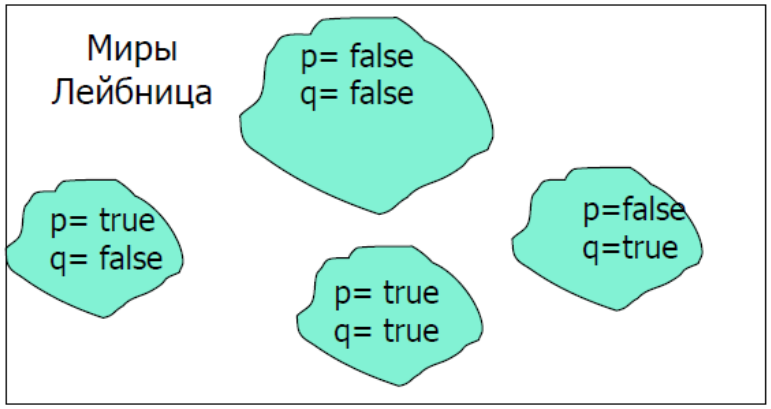


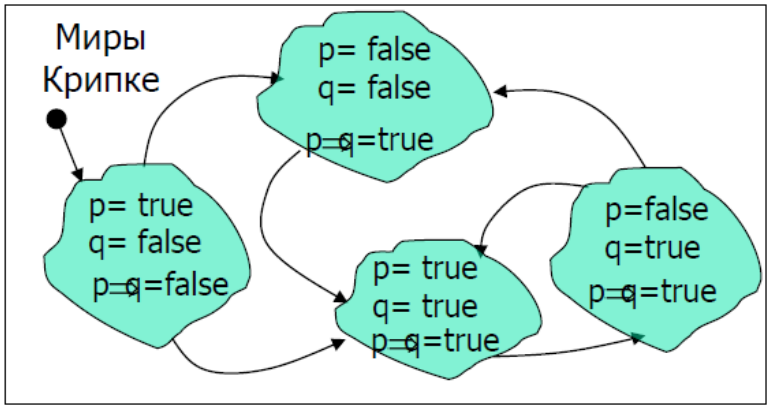
Структуры Крипке



Структуры Крипке. Что такое “истина”?

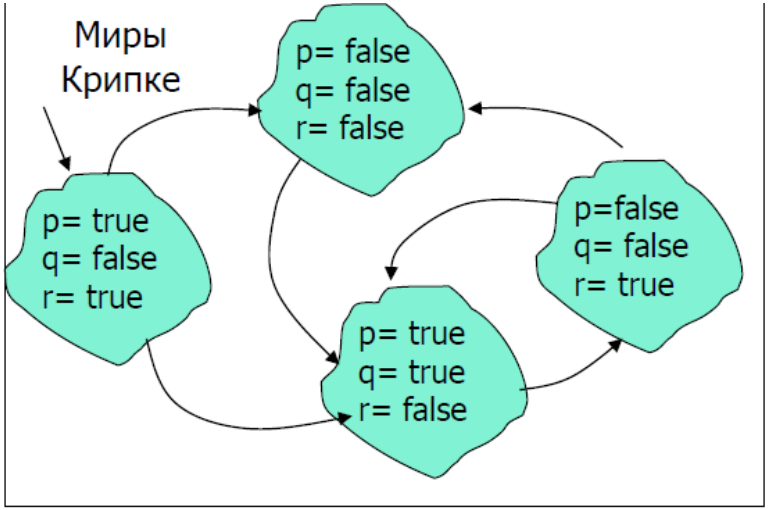
- Лейбниц ввел понятия
  - необходимых” истинных утверждений, которые всегда истинны (например, 2×2=4, “Земля вертится”, и т.д.), И
  - “возможных” истинных утверждений, истинность которых в некоторых ситуациях возможна, а в некоторых – нет (например, “Я люблю Дашу”, “Светит солнце”, ...).
- Например, “идет дождь” – истинно в ситуации, когда действительно идет дождь. “Возможные” истинные утверждения истинны в каких-то специальных, “возможных” мирах. В каждом мире свой набор истинностных значений атомов, и поэтому каждое сложное высказывание либо истинно, либо ложно в каждом мире
- Семантика ППФ определяется как истинностное значение (true или false) этой формулы в каждом возможном мире

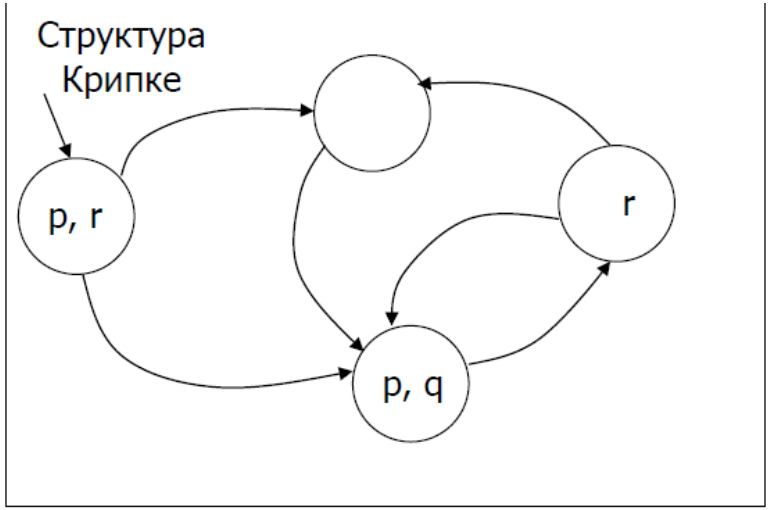




- Сол Крипке (американский логик и философ) ввел отношение достижимости на множестве миров, которое можно трактовать по-разному, например, как переход во времени

Структуры Крипке



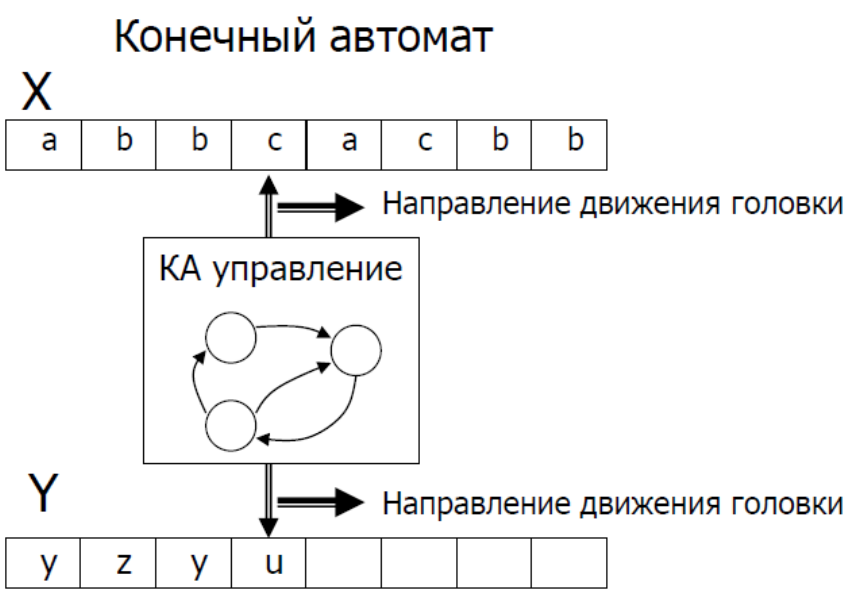


- Структура Крипке – это помеченная система переходов, с каждым состоянием которой связаны атомарные утверждения, истинные в этом состоянии
- СК используется для представления СЕМАНТИКИ ВОЗМОЖНЫХ МИРОВ в методе логического анализа понятий, основу которого составляет рассмотрение мыслимых положений дел (идеальных альтернатив, описаний состояний, точек соотнесения)
- СК используется как модель реагирующих систем (reactive systems) при верификации
- При представлении программ структурами Крипке атомарные утверждения выбираются программистом: p='x>2*y'; q='буфер пуст'; r='ответ получен'

МП – автоматы и Машины Тьюринга



Конечные автоматы vs Автоматы с магазинной памятью (МП-автоматы)



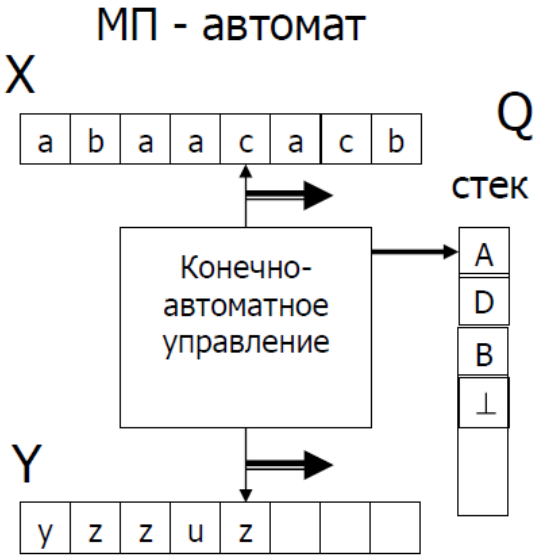
$A=(S, X, Y, s_0, \delta)$

$\delta: S \times X \to S \times Y$

Программа

1. $(s,a) \to (q, y)$
2. $(p,b) \to (s,z)$
3. ...

В МП-автомате к КА добавляется бесконечная память - стек



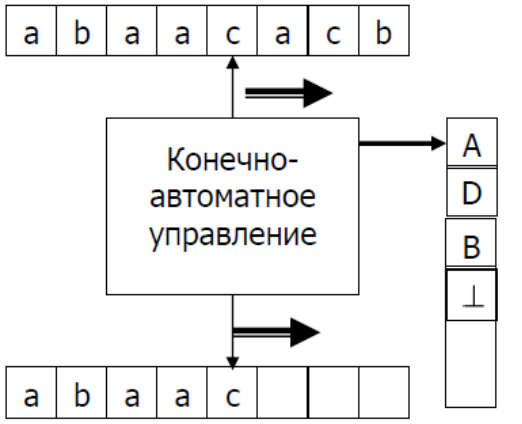
$M=(S, X, Y, Q, s_0, \delta)$

$\delta: S \times X \times Q \to S \times Y \times Q^*$

Программа

1. $(s,a,\varepsilon) \to (q, y, AB)$
2. $(p,b,A) \to (s, z, \varepsilon)$
3. ...

Автоматы с магазинной памятью. Пример

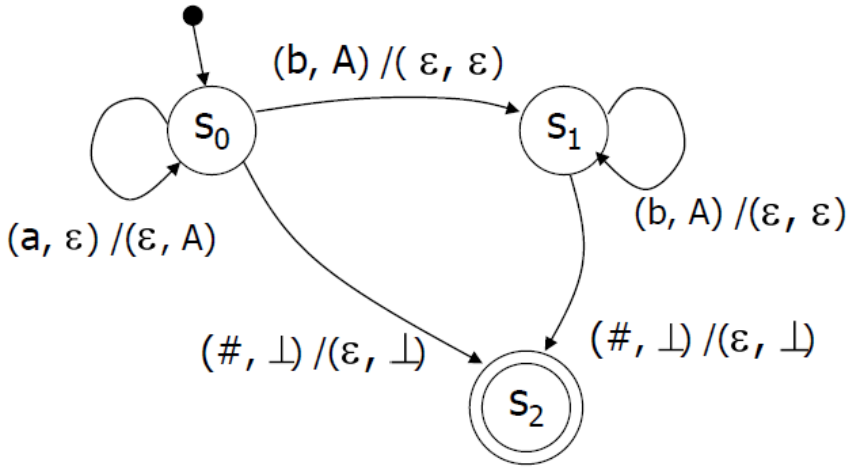


$M=(S, X, Y, Q, S_0, \delta)$

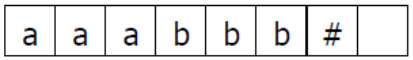
$\delta: S \times X \times Q \rightarrow S \times Y \times Q^*$

$(s_0,a,\varepsilon) \rightarrow (s_1,y_3,AB)$

Пример магазинного автомата: распознаватель языка $a^n b^n$

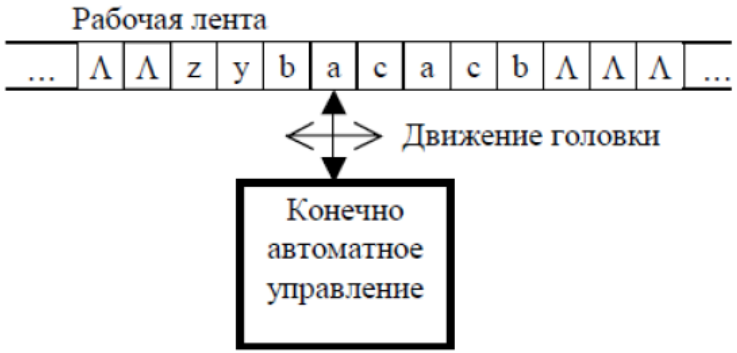


Пример распознаваемой входной цепочки

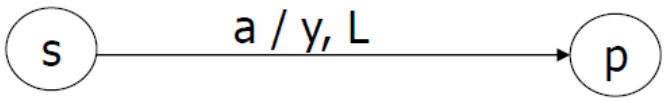


Машины Тьюринга

Алан Тьюринг в 1936г. предложил формальную модель вычислителя, которая является результатом простого добавления потенциально бесконечной памяти к системе управления с конечной памятью



Программа МТ – конечное множество пятерок
<s, a, p, y, D>, или
(s, a) ⟶ (p, y, D)



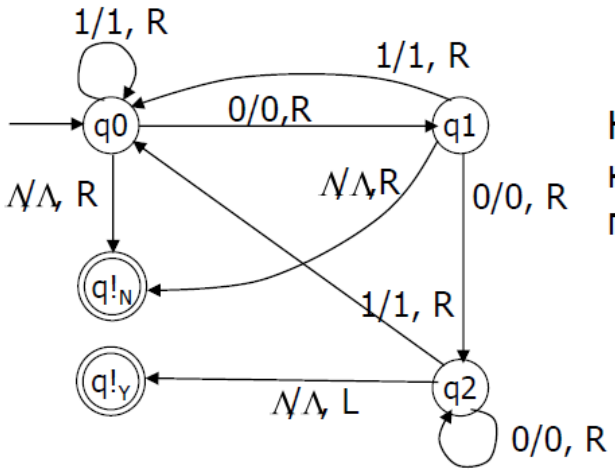
МТ = (S, Σ, s_0, δ, Г)
Г = {L, R, H};
δ: S × Σ ⟶ S × Σ × Г

Программа:
1. (s, a) ⟶ (p, y, L);
2. (q, b) ⟶ (s, z, R);
3. …

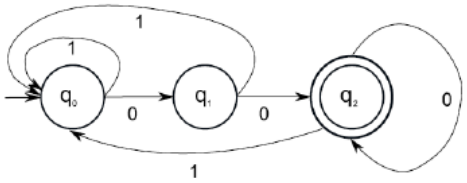
S – состояния
Σ - вход. алфавит
s_0 – начальное состояние
δ – функция переходов
Г – движение головки

Команды:
(состояние, вход) → (состояние, выход, D∈Г)

Машина Тьюринга, распознающая язык цепочек из 0 и 1, заканчивающихся на 00

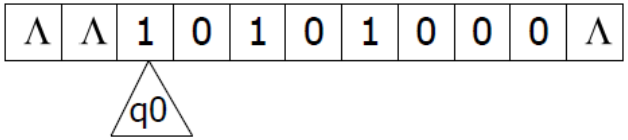


Это автоматный язык!



Начальная конфигурация МТ: МТ находится в начальном состоянии q0, головка расположена против крайнего левого символа входной цепочки

Начальная конфигурация МТ:



К конечному автомату добавлена потенциально бесконечная внешняя лента

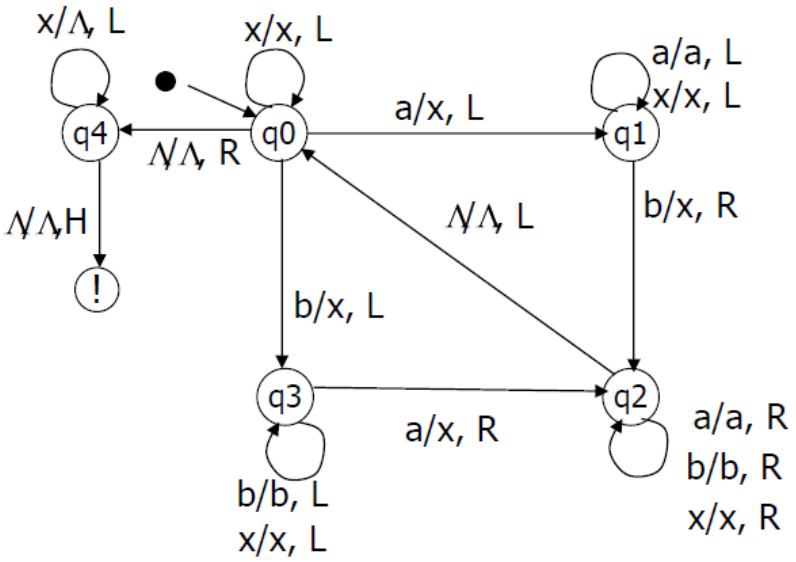
Полученная машина Тьюринга представляет собой универсальный преобразователь информации

	0	1	Λ
q0	(0,q1,R)	(1,q0,R)	(Λ,q!$_N$,R)
q1	(0,q2,R)	(1,q0,R)	(Λ,q!$_N$,R)
q2	(0,q2,R)	(1,q0,R)	(Λ,q!$_Y$,L)

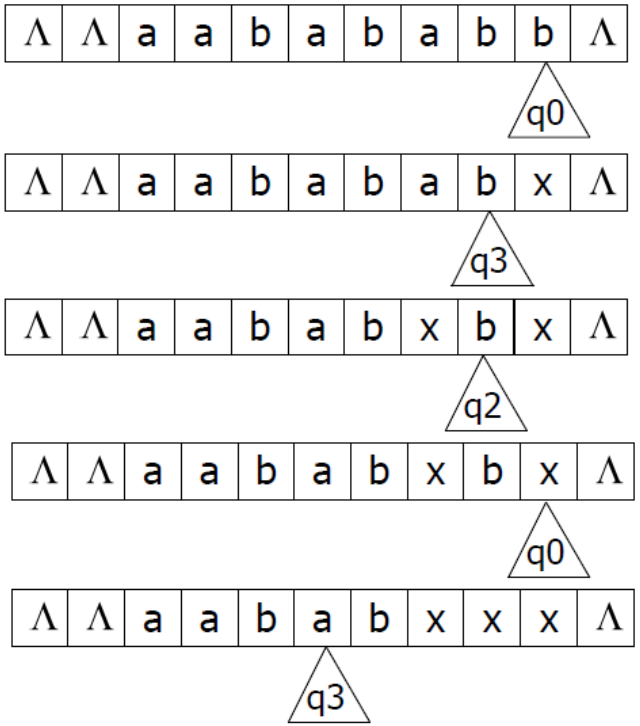
Машина Тьюринга, распознающая язык с одинаковым числом вхождений символов *a* и *b*

Начальная конфигурация МТ: МТ находится в начальном состоянии q0, а головка расположена против крайнего правого символа входной цепочки).

Это НЕ автоматный язык!



начальная конфигурация:

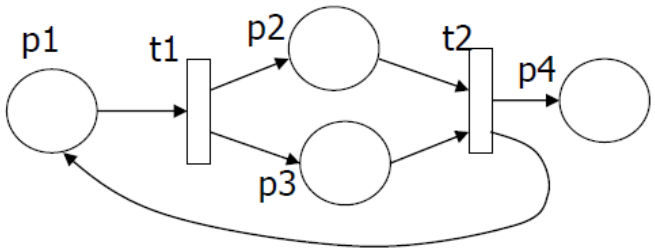


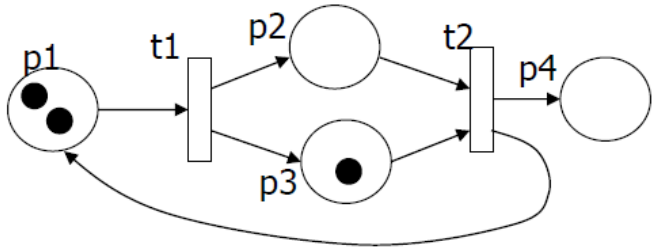
Сети Петри



Сети Петри

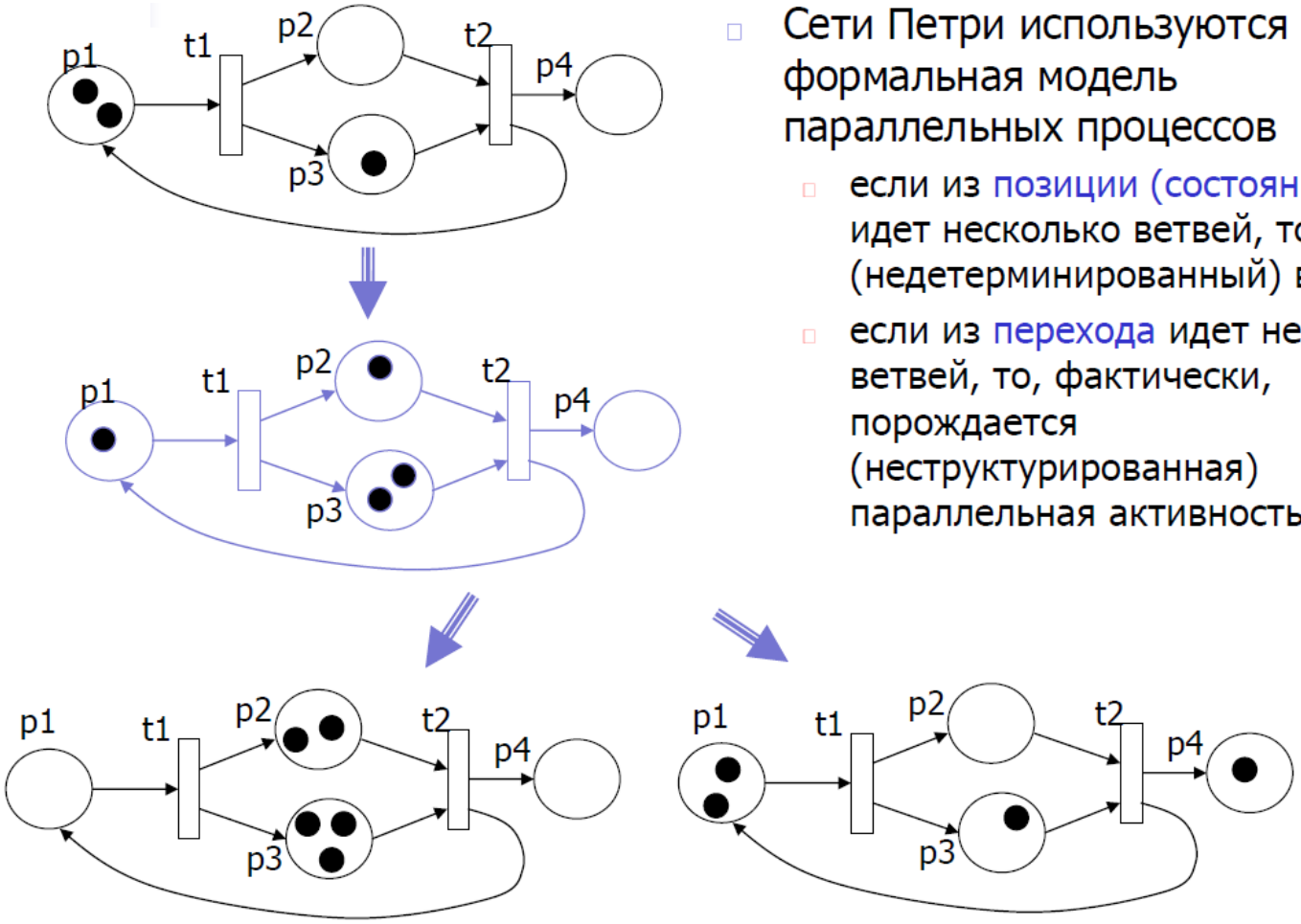
- Формальная модель, используемая для задания поведения дискретных динамических систем. Впервые описаны Карлом Адамом Петри в 1962 г.
- Сеть Петри – это двудольный ориентированный граф, состоящий из вершин двух типов — позиций и переходов, соединённых между собой дугами. В позициях могут размещаться метки (маркеры), способные перемещаться по сети





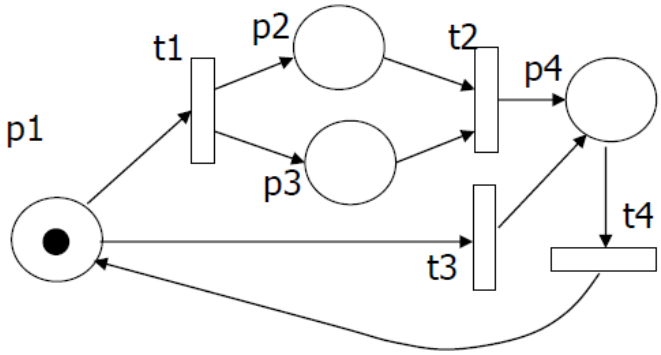
- Правила срабатывания перехода:
  - переход возбуждается (fires), если в каждой из входящих в него позиций есть не менее одного маркера
  - если возбуждены несколько переходов, срабатывает только один (любой) из них
  - при срабатывании перехода, из каждой позиции, связанной с переходом входящей в переход дугой, изымается по одному маркеру, и в каждую позицию, в которую входят выходящие из перехода дуги, маркер добавляется

Пример сети Петри



- Сети Петри используются как формальная модель параллельных процессов
  - если из позиции (состояния) идет несколько ветвей, то это (недетерминированный) выбор
  - если из перехода идет несколько ветвей, то, фактически, порождается (неструктурированная) параллельная активность

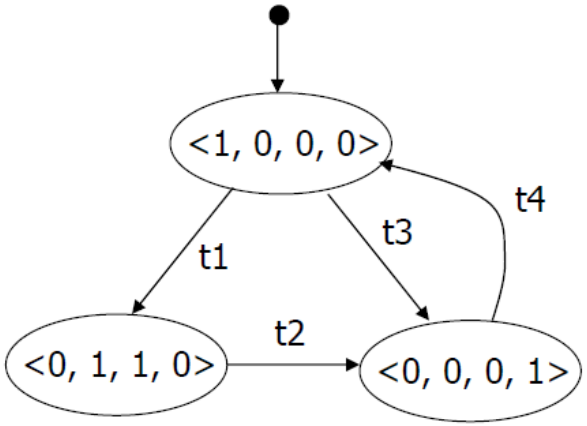
Сети Петри и системы переходов (графы достижимых маркировок)



Состояние СП – число фишек в позициях

Начальное состояние <1, 0, 0, 0>

Граф достижимых маркировок этой СП – это система переходов (конечное число состояний и переходов) :

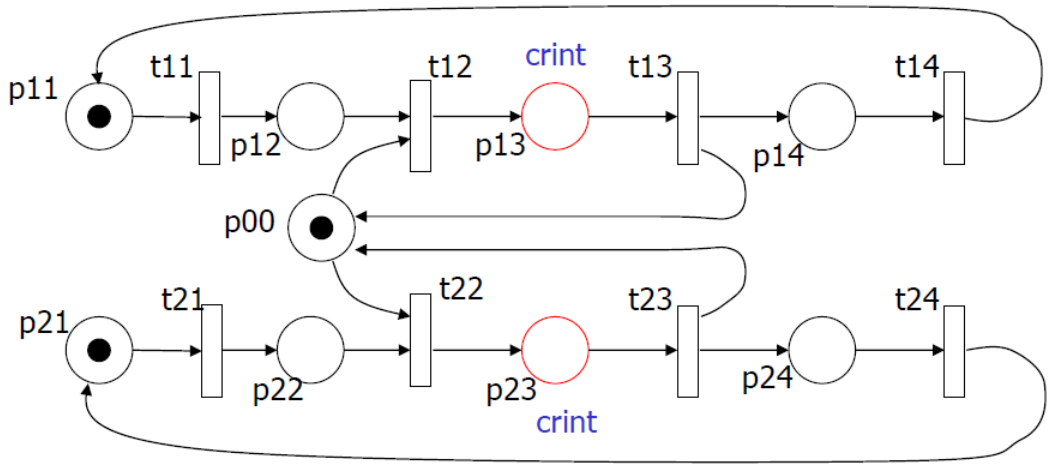


Свойства сетей Петри

По графу достижимых маркировок можно проанализировать все свойства СП

Основные свойства сети Петри:

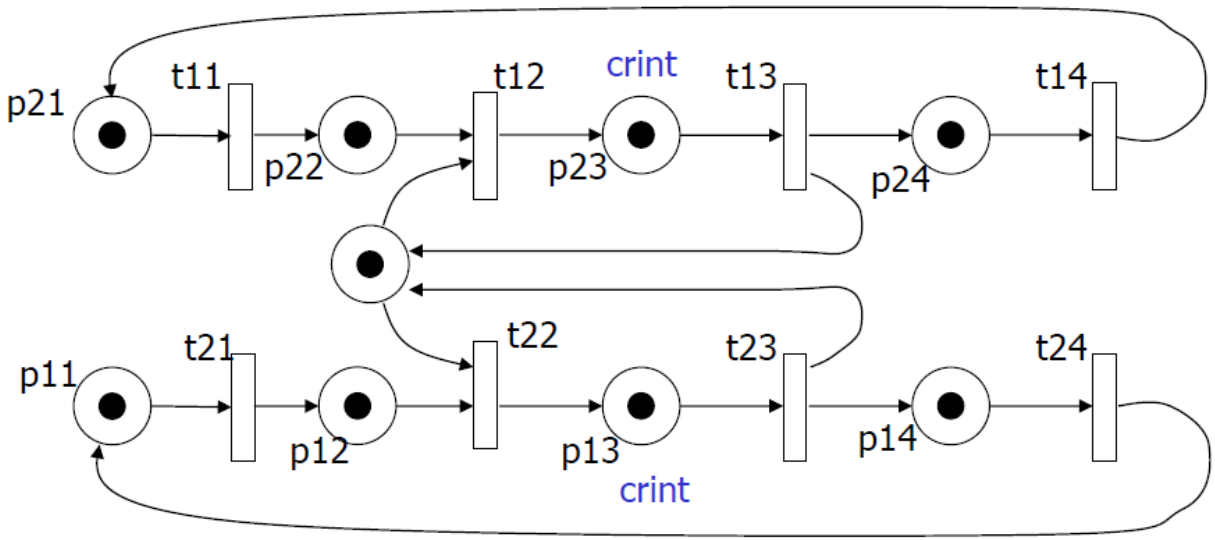
- ограниченность — число меток в любой позиции сети не может превысить некоторого значения K;
- безопасность — частный случай ограниченности, K=1;
- достижимость — возможность перехода сети из одного состояния (обычно из начального) в заданное состояние;
- живость — возможность срабатывания любого перехода при функционировании СП (иногда – отсутствие блокировки, останова)



Пример: синхронизация доступа параллельных процессов к общему ресурсу

Свойства сетей Петри

Пример: синхронизация доступа параллельных процессов к общему ресурсу



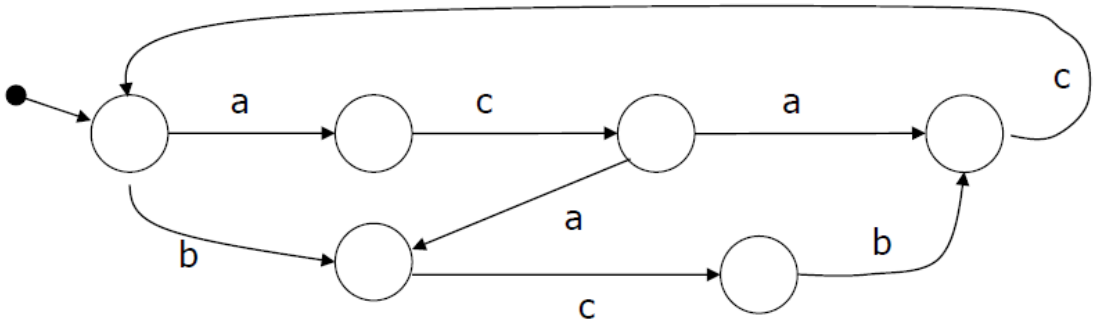
Эта сеть:
- ограничена
- безопасна
- жива
- некорректное состояние недостижимо

Конечные автоматы являются частным случаем сетей Петри

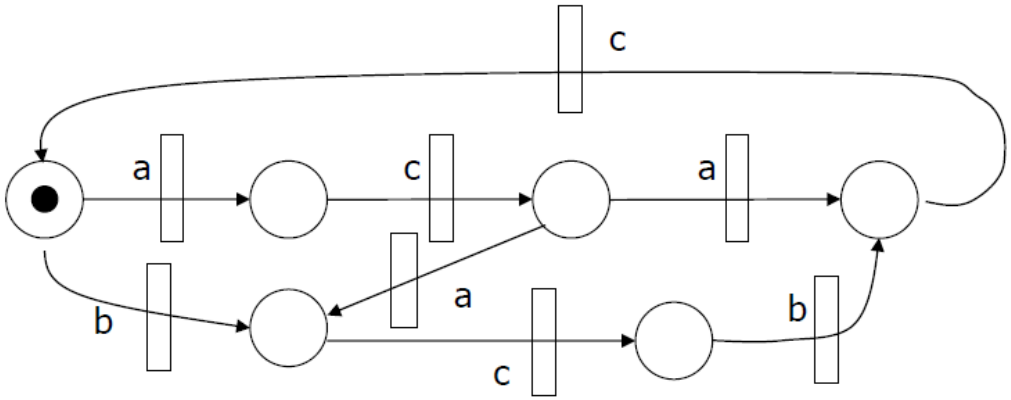
Конечные автоматы как подкласс сетей Петри

Пример: Если у переходов сети Петри только одна входная и одна выходная стрелки, и только один маркер в некоторой позиции, то эта сеть Петри является КА

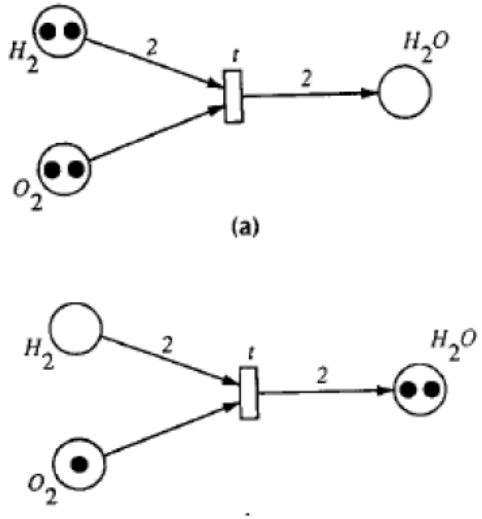
КА



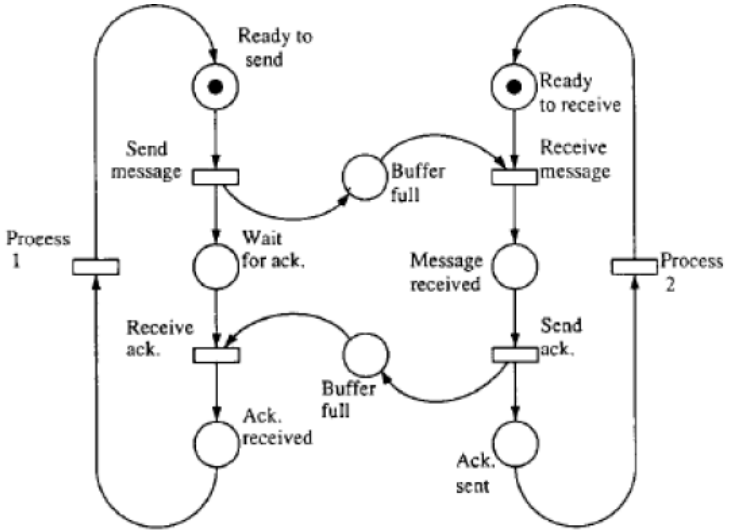
Сеть Петри



Примеры применения Сетей Петри



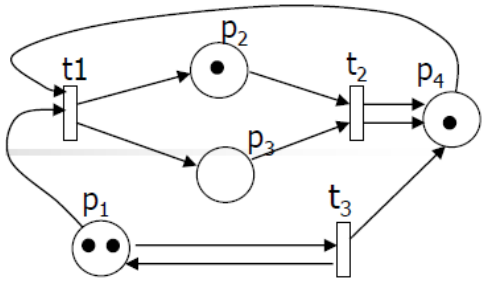
Моделирование химических реакций



Моделирование простого протокола передачи данных через буфер

- Сети Петри можно использовать для формализации и визуализации динамических процессов любой природы

Формализм: сети Петри



- Сеть Петри (СП) – это пара (S, μ_0), где
  S – некоторая сеть,
  μ_0 –начальная разметка сети S
- Сетью называется двудольный граф S = (P, T, F), где
  P и T – непустые непересекающиеся конечные множества, называемые местами и переходами;
  $F \subseteq (P \times T) \cup (T \times P)$ – бинарное отношение инцидентности
- для любого элемента $x \in P \cup T$ отношение F определяет множества его входных и выходных элементов:
  $^\bullet x = \{y | yFx\}$
  $x^\bullet = \{y | xFy\}$
- Разметкой (маркировкой) сети S = (P, T, F) называется функция $m: P \to N$. Если $m(p)=k$, то в месте p имеется k фишек
- Переход $t \in T$ допустим на разметке μ СП (S, μ_0) тогда $(\forall p \in {}^\bullet t)$: $m(p) \geq F(p, t)$

Переход t_1 допустим на разметке <2, 3, 0, 1>

Переход t_2 НЕ допустим на разметке <2, 3, 0, 1>

$P=\{p_1, p_2, p_3, p_4\}$ $T=\{t_1, t_2, t_3\}$

$F=\{ (p_1,t_1), (p_2,t_2), (p_3,t_2), (p_3,t_2), (p_1,t_3), (p_4,t_1) \} \cup \{(t_1,p_2),(t_1,p_3),(t_2,p_4), (t_3,p_4), (t_3,p_1)\}$

$F(p_1,t_1)=1$, $F(p_1,t_2)=0$, $F(p_1,t_3)=1$, ...

$F(t_1,p_1)=0$, $F(t_2,p_2)=0$, $F(t_2,p_4)=2$, ...

$^\bullet p_1=\{t_3\}$; $^\bullet p_2=\{t_1\}$; $^\bullet p_3=\{t_1\}$;
$^\bullet p_4=\{t_2\}$; $p_1^\bullet=\{t_1\}$; $p_2^\bullet=\{t_1, t_2\}$;
$p_3^\bullet=\{t_2\}$; $p_4^\bullet=\{t_1\}$;

$^\bullet t_1=\{p_1,p_4\}$; $^\bullet t_2=\{p_2,p_3\}$; $^\bullet t_3=\{p_1\}$;
$t_1^\bullet=\{p_2, p_3\}$; $t_2^\bullet=\{p_4\}$; $t_3^\bullet=\{p_1,p_4\}$;

$\mu_0=<2,1,0,1>$

Взаимодействующие конечные автоматы

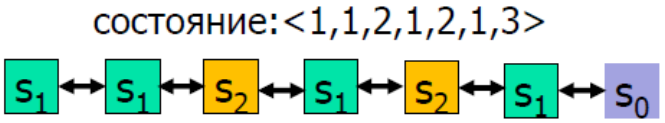


Взаимодействующие конечные автоматы

- В последнее время развивается теория работы дискретных устройств, т. е таких устройств, которые состоят из большого числа отдельных элементов и работают отдельными тактами
- Каждый элемент такой системы может находиться в небольшом числе состояний (т.е. является конечным автоматом), и изменение состояния отдельного элемента может зависеть как от его предыдущего состояния, так и от предыдущих состояний обычно небольшого числа “соседних”, связанных с ним элементов. Так устроены компьютеры, так, предположительно, устроен и человеческий мозг
- Одной из простейших моделей такого типа является клеточный автомат



Клеточные автоматы



- Клеточные автоматы – это регулярная структура динамических объектов (клеток), функционирующих синхронно. Клеточные автоматы моделируют процессы, разворачивающиеся в дискретном пространстве и в дискретном времени
- Клеточный автомат имеет состояние, определяемое как набор (вектор или матрица) состояний компонентных автоматов. Каждый автомат функционирует как автомат Мура, т.е. это автомат без выхода, выходом является его состояние. Вход на каждом шаге клеточного автомата – состояния его соседей. На следующем шаге (в следующем такте) новое состояние каждого автомата определяется как функция его текущего состояния и текущих состояний его соседей
- Обычно правила изменения состояний всех автоматов (кроме крайних) идентичны (однородность системы), множество состояний каждого автомата – конечно. Изменение состояния системы во всех клетках происходит синхронно: состояния клеток меняются одновременно, в каждом такте
- **Каждое состояние – своим цветом. Тогда динамика – просто картинка**

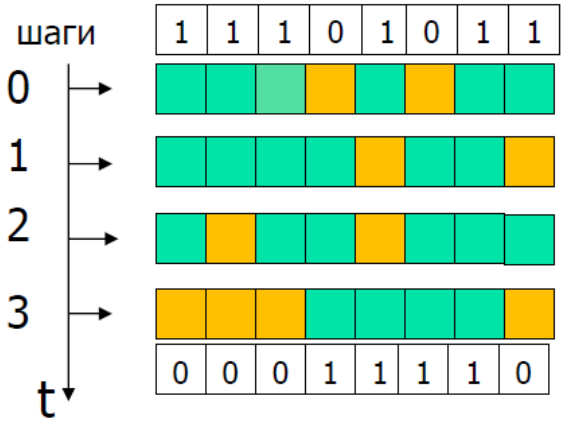
А.Н.Колмогоров: "Системы из очень большого числа элементов, каждый из которых действует чисто "арифметически", могут приобретать качественно новые свойства"

Марвин Минский: "важным ... является изучение ... возникновения сложного поведения из простых устройств, действий, описаний или концепций"

Одномерная сеть клеточных автоматов



8 автоматов, выстроенных в линию



На картинке – четыре дискретных момента времени, 8 автоматов с двумя состояниями, выстроенных в линию

Задача: придумать простые правила, которые порождают интересные картинки

Задача: посмотреть, какие картинки порождаются различными правилами

Простейшая модель: всего 2 состояния, и состояние новое (на следующем такте) есть функция $f(s_{левого}, s_{свое}, s_{правого})$ – текущих состояний

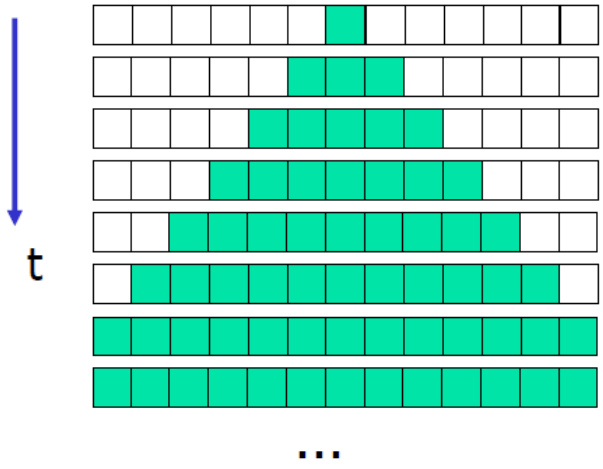
При двух возможных состояниях все сводится к двоичной логике, и новое состояние – это двоичная функция от трех двоичных переменных

Удобство клеточных автоматов – то, что их поведение может быть представлено визуально

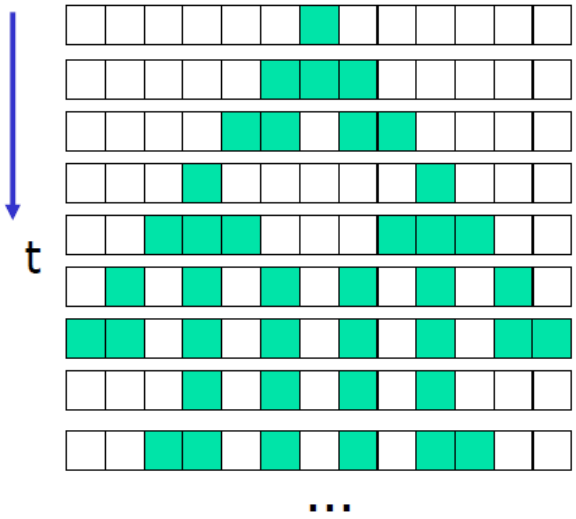
Клеточные автоматы. Одномерная структура

- Простейшее число состояний – 2. Эти состояния можно понимать как состояния “живая” и “мертвая” клетки. На каждом шаге состояние каждой клетки (1 или 0) есть булева функция от 3-х переменных (ее самой и двух соседей): $s_i' = \delta(s_{i-1}, s_i, s_{i+1})$.
- Динамика (поведение) определяется начальным состоянием линейки и этой функцией переходов (всего таких функций 256).

Пример:
$\delta(\text{Left}, s, \text{Right}) = \text{Left} \vee s \vee \text{Right}$

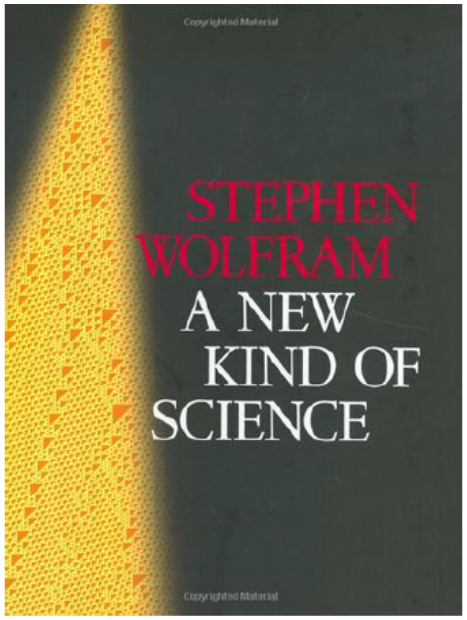


Пример: $\delta(L,s,R) = L \oplus s \oplus R$



Одномерные клеточные автоматы: поведение во времени

Клеточные автоматы. Новая наука Стефена Вольфрама



Стефен Вольфрам описывает ряд "драматических открытий", сделанных им с помощью простых компьютерных программ, являющихся правилами поведения клеточных автоматов

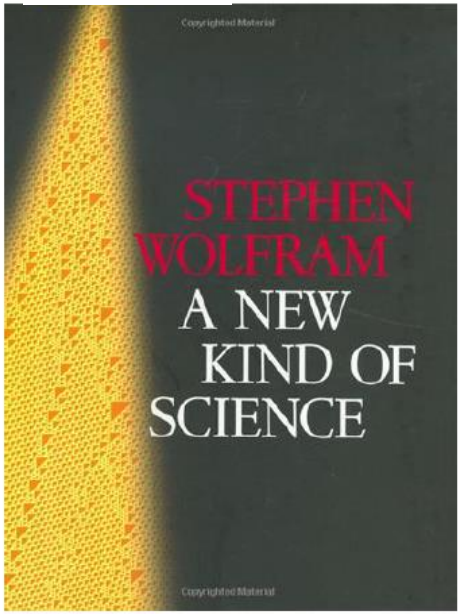
Он показывает, как совокупности взаимодействующих простейших программ способны порождать сложнейшие клеточные системы, сравнимые по сложности с Универсумом

Вольфрам убежден, что можно построить клеточные автоматы с простыми правилами функционирования в качестве моделей фундаментальных концепций физики, биологии и философии

"Такие системы простых взаимодействующих конечных систем с памятью способствуют рождению *новой научной интуиции*, новому пониманию фундаментальных проблем бытия"

Вольфрам считает, что они открывают путь к "новой науке"

Стефен Вольфрам – книга о новом типе науки



Из рецензии: Это давно ожидавшаяся книга одного из наиболее компетентных ученых представляет множество драматических открытий. Начиная с простейших компьютерных экспериментов Wolfram показывает, как эти неожиданные результаты прокладывают новый путь в познании Вселенной

"This is more than just a book: it is a phenomenon, a paradigm shift...."
—New Scientist

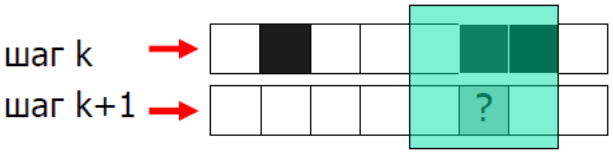
"A New Kind of Science *is a book that simply cannot be ignored.... Such a book appears only once every few decades."*
—Nature

"There is so much in the book that it will be years, literally years, before people assimilate it."
—The New York Times

"Stephen's magnum opus may be the book of the decade, if not the century."
—Arthur C. Clarke

Нумерация функций по Вольфраму

Аргументы – состояние клетки и двух соседей, слева-справа

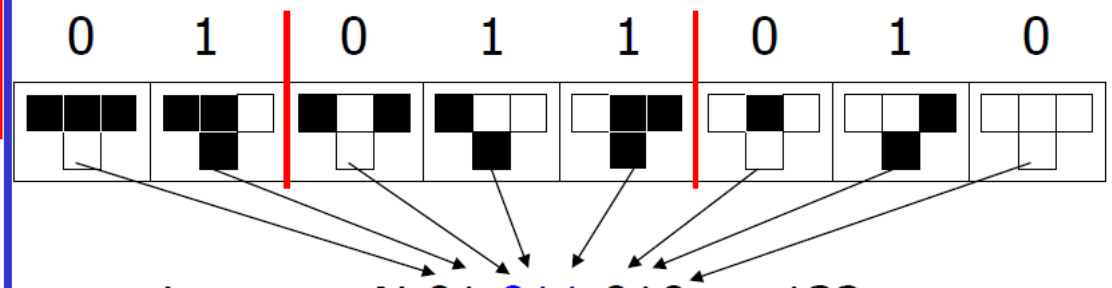


? = F(L, s, R) = F(□, ■, ■)

Двоичная функция

Функция изменения состояний

L s R	F
0 0 0	0
0 0 1	1
0 1 0	0
0 1 1	1
1 0 0	1
1 0 1	0
1 1 0	1
1 1 1	0



Функция N $01\,011\,010_2 = 132_8$

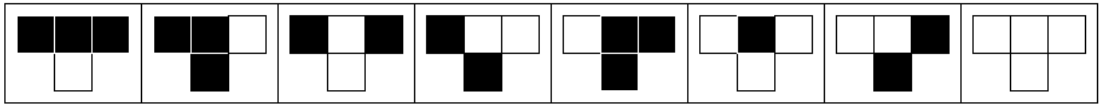
$1*8^2 = 64$ | $3*8^1 = 24$ | $2*8^0 = 2$

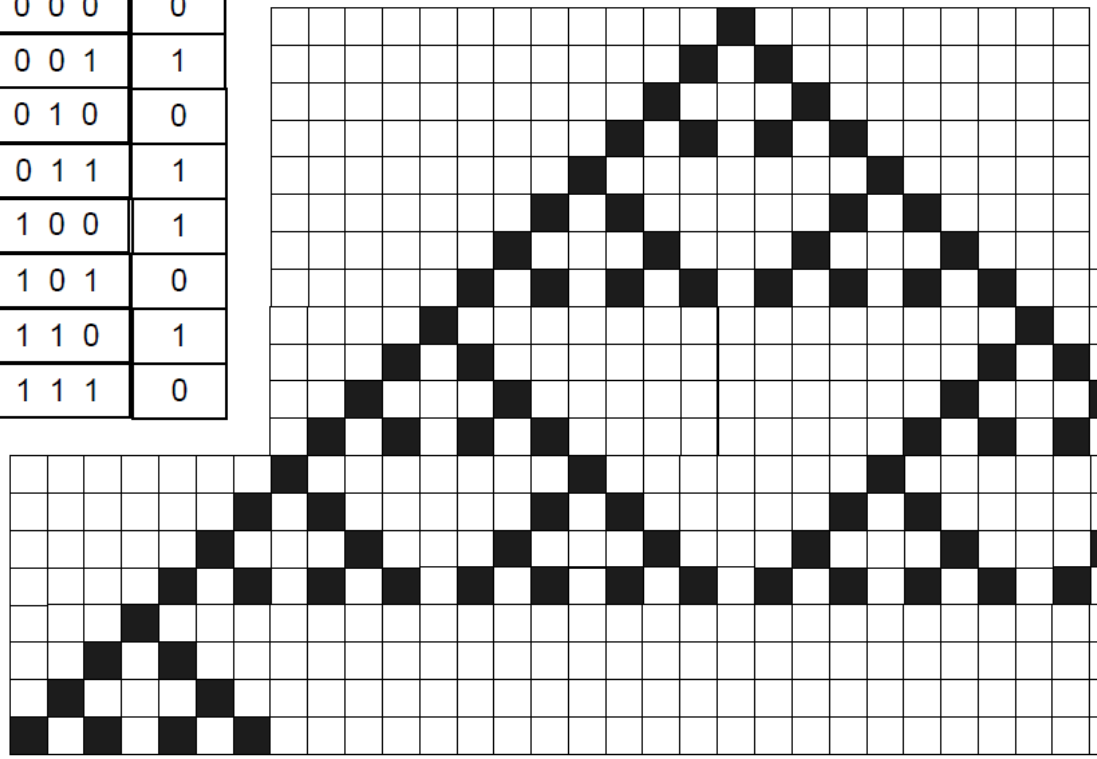
$64 + 24 + 2 = 90_{10}$

Функция N 90_{10}

Поведение функции 90

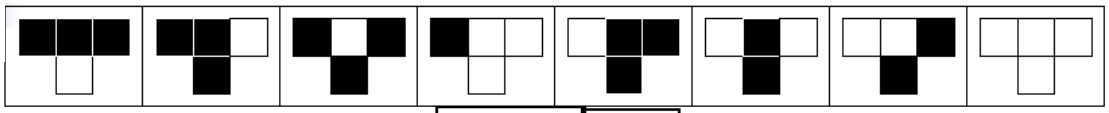
x y z	F_{90}
0 0 0	0
0 0 1	1
0 1 0	0
0 1 1	1
1 0 0	1
1 0 1	0
1 1 0	1
1 1 1	0





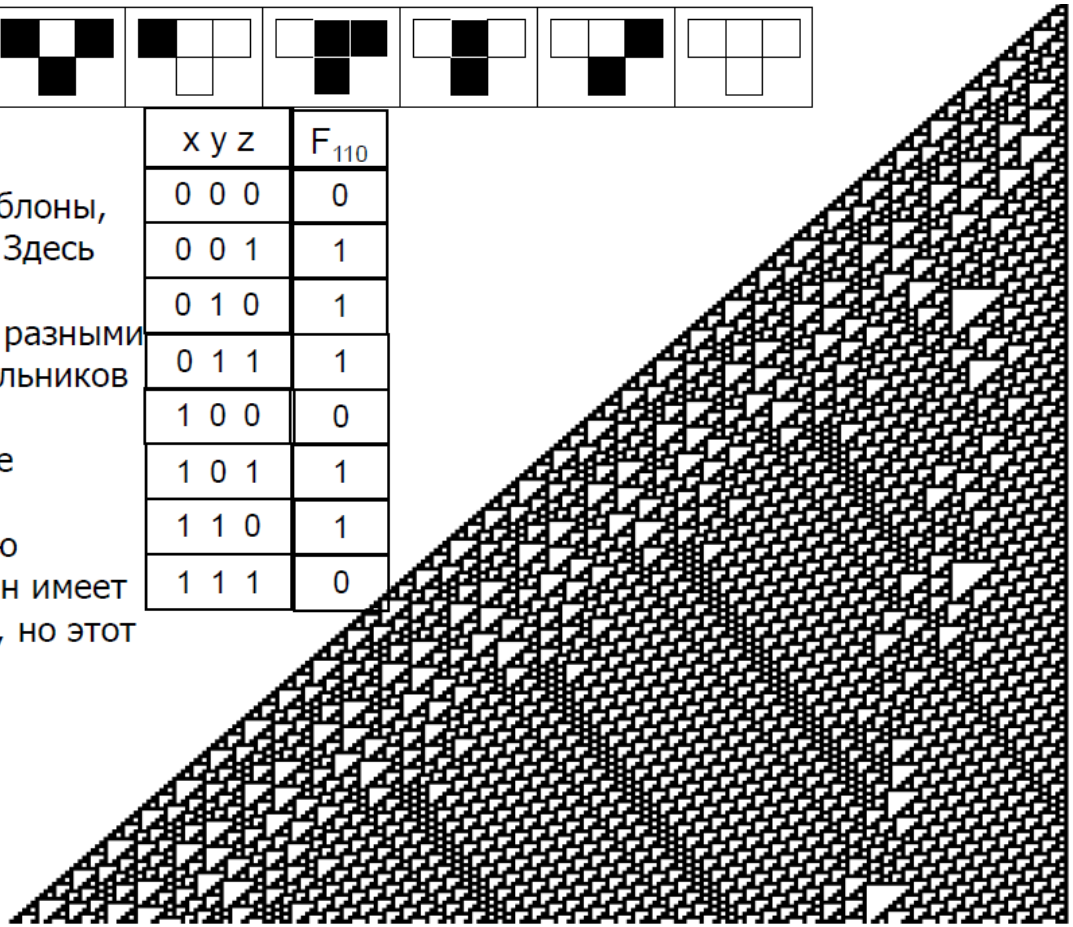
Для различных функций получаются разные фигуры

Булева функция 110

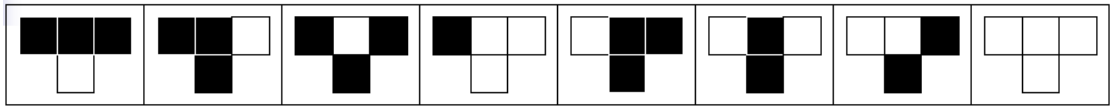


x y z	F_{110}
0 0 0	0
0 0 1	1
0 1 0	1
0 1 1	1
1 0 0	0
1 0 1	1
1 1 0	1
1 1 1	0

- Функция 110 генерирует удивительно сложные шаблоны, которые не повторяются. Здесь присутствует огромное разнообразие: линии под разными углами, скопления треугольников и другие конфигурации. Получающийся рисунок не является ни полностью регулярным, ни полностью случайным, хаотичным. Он имеет свой внутренний порядок, но этот порядок непредсказуем.
- С.Вольфрам считает, что функция 110 является истинным источником сложности мира



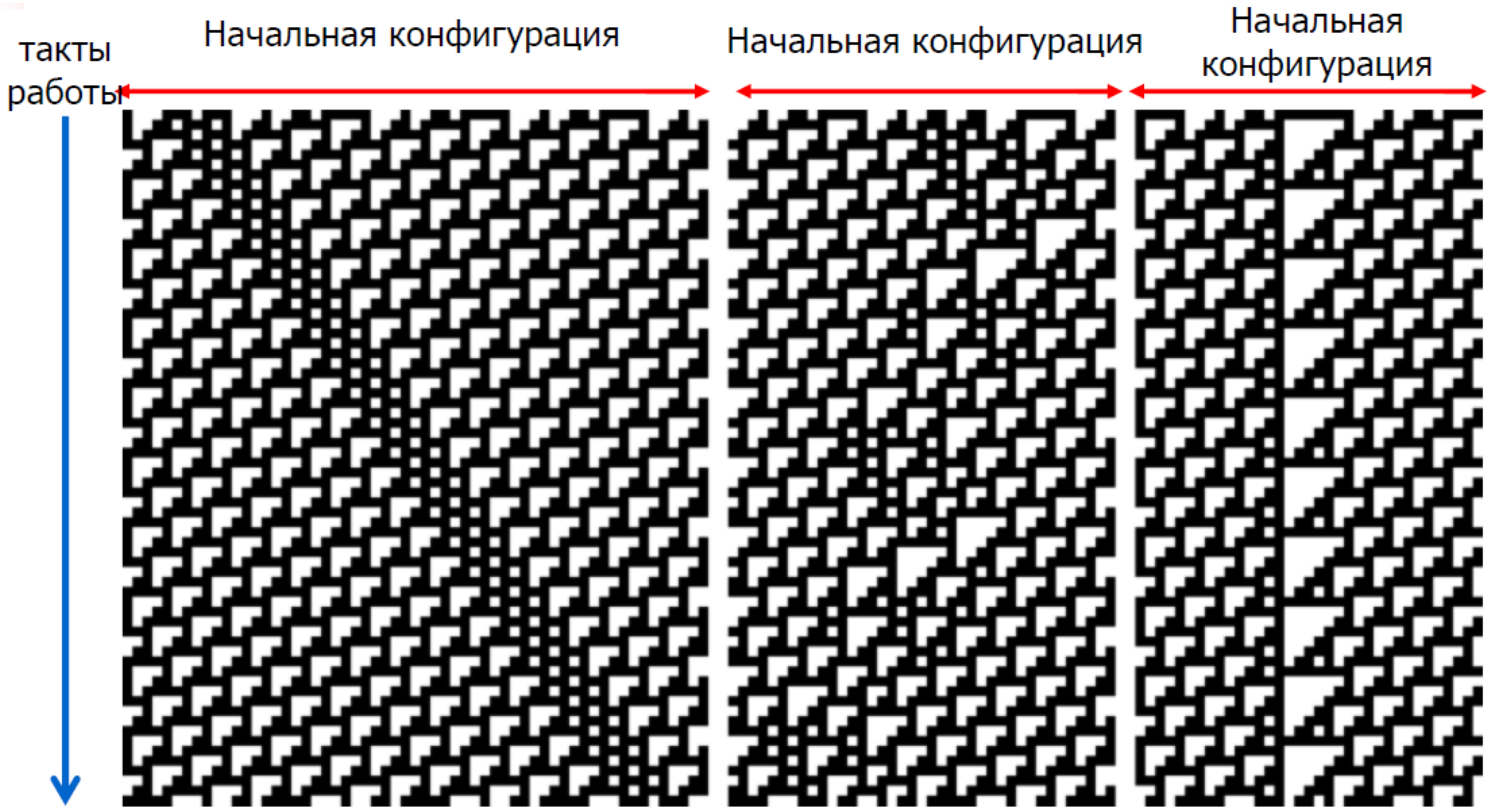
Rule 110 CA – универсальный вычислитель



x y z	F_{110}
0 0 0	0
0 0 1	1
0 1 0	1
0 1 1	1
1 0 0	0
1 0 1	1
1 1 0	1
1 1 1	0

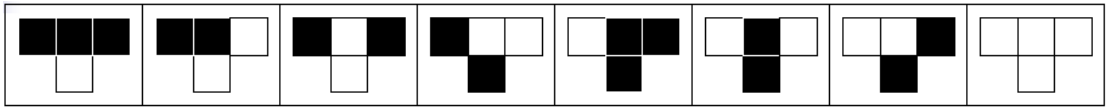
- Из Википедии:
- The **Rule 110 cellular automaton** (often simply **Rule 110**) is an elementary cellular automaton with interesting behavior on the boundary between stability and chaos. In this respect it is similar to Game of Life. Rule 110 is known to be Turing complete. This implies that, in principle, any calculation or computer program can be simulated using this automaton
- Around 2000, Matthew Cook published a proof that Rule 110 is Turing complete, i.e., capable of universal computation, which Stephen Wolfram had conjectured in 1985. Rule 110 is arguably the simplest known Turing complete system.[1]

Фигуры, получающиеся в результате работы линейки клеточных автоматов 110

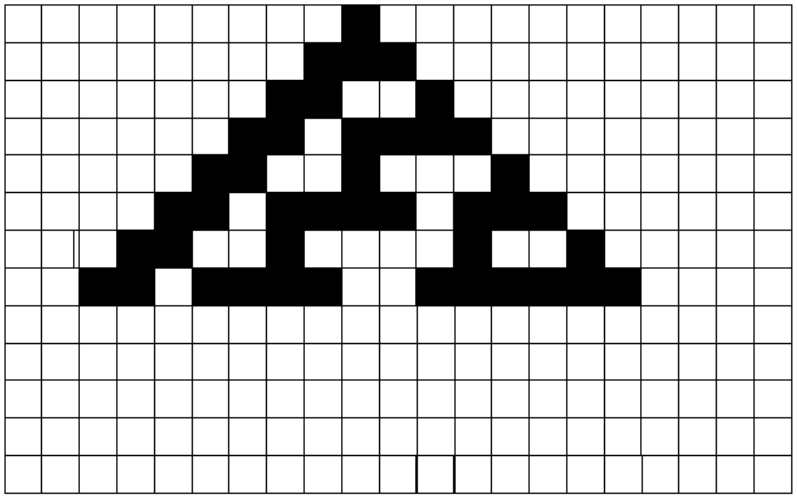


- Клеточный автомат 110 может моделировать машину Тьюринга, которая, в свою очередь, может выполнять какие угодно вычисления.

Поведение функции 30

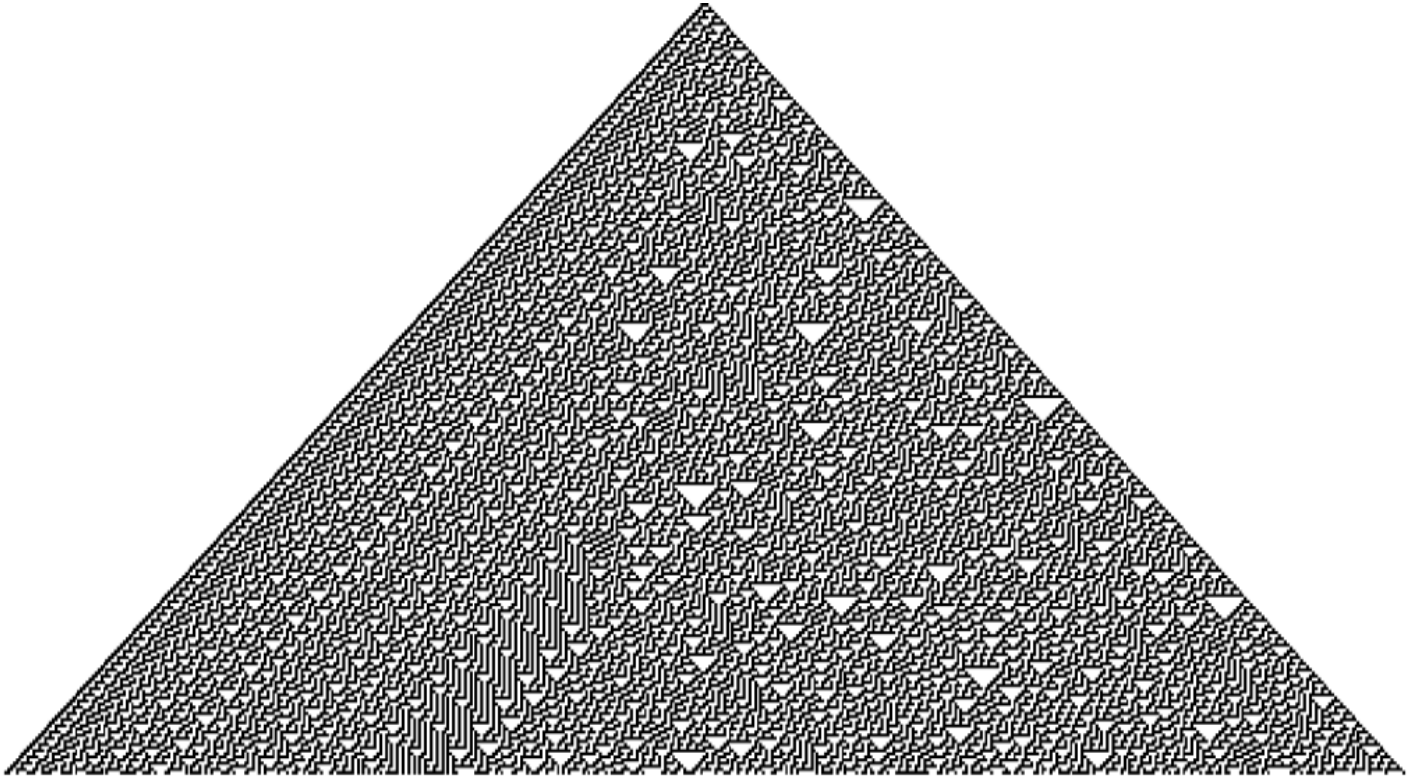


x y z	F_{30}
0 0 0	0
0 0 1	1
0 1 0	1
0 1 1	1
1 0 0	1
1 0 1	0
1 1 0	0
1 1 1	0



Для различных функций получаются разные фигуры

Функция 30

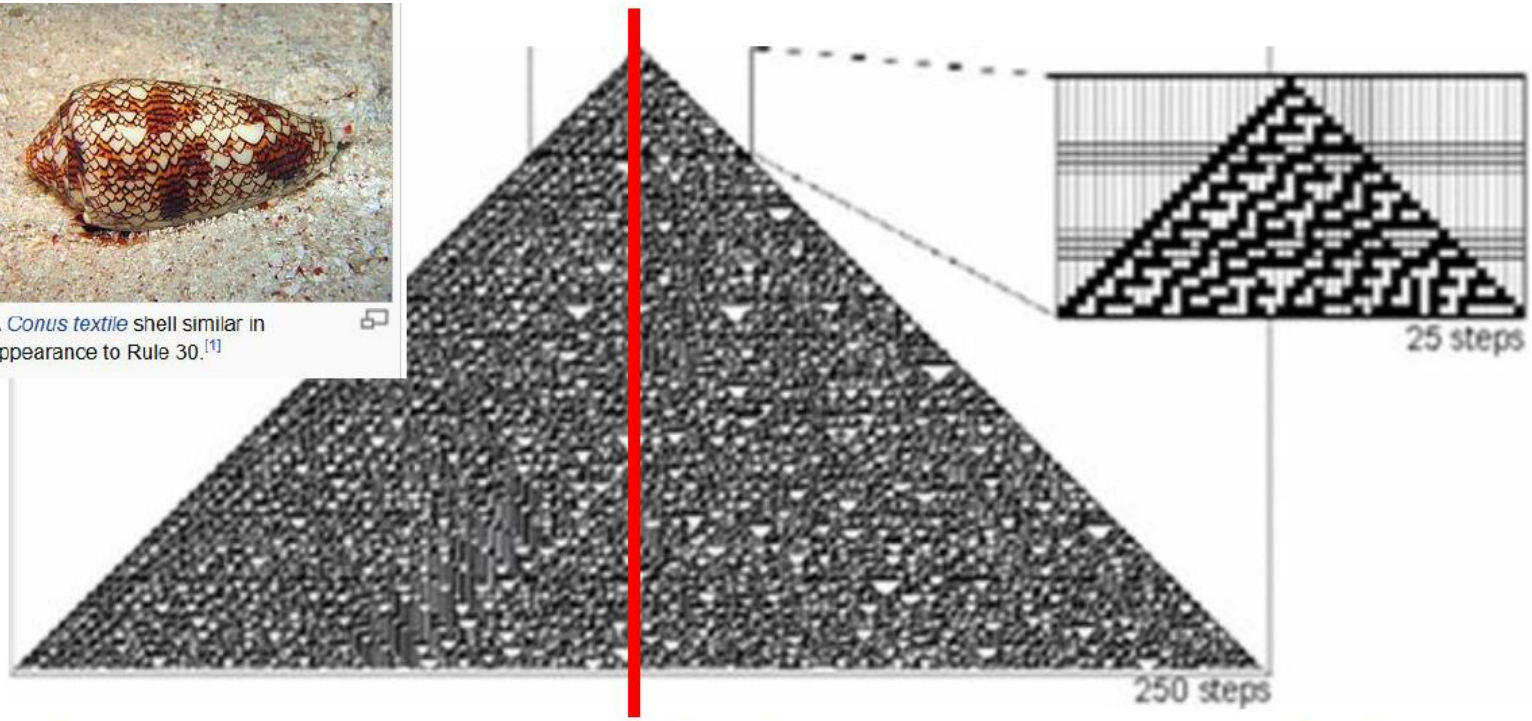


Rule 30 cellular automaton

current pattern	111	110	101	100	011	010	001	000
new state for center cell	0	0	0	1	1	1	1	0

Поведение функции 30

A *Conus textile* shell similar in appearance to Rule 30.[1]



Хаотическая сложная структура. Вольфрам проверил, что цепочка 0 и 1 по центральной вертикали является статистически случайной!

Гипотеза: сложное есть композиция очень многих очень простых

Сложное поведение как композиция простейших!!!

Можно исследовать динамику одномерного КлА не только с единственной “живой” клеткой, но и с другими начальными условиями, не только одномерную, но и двумерную, трехмерную структуры, число состояний можно увеличивать (сделать >2), правила можно сделать не только детерминированными, не только локальными, и т.п.

Некоторые расширения приведены в книге Вольфрама на 1200 страницах. В книге множество впечатляющих примеров!!!

- В книге Вольфрама звучит призыв – не нужно решать дифуры, перемножать матрицы, делать приближения. Вместо этого нужно генерировать структуры, анализировать их макроскопические характеристики и визуальные узоры. Это даст новый взгляд на суть вещей, и, возможно, приведет к решению не решенных ранее проблем. Решения некоторых таких проблем - в книге A New Kind of Science

Вольфрам: Все, что потенциально может существовать в природе, может быть получено из очень простых строительных блоков, которые взаимодействуют локально друг с другом. Из множества простейших элементов нижнего уровня, взаимодействующих только со своими соседями, могут моделироваться сложные физические системы: жидкости, оптика, динамика населения, и, возможно, даже сама природа пространства-времени

- Книга Вольфрама вызвала широкую дискуссию. Часть ученых считает, что это – профанация, часть ученых считает, что в этом что-то есть, очень многие с восторгом восприняли этот новый взгляд



Самоорганизующиеся системы

Повторяющиеся простые преобразования могут приводить в результате к очень сложным явлениям, и, в соответствии с идеями Вольфрама, именно взаимодействующие очень простые алгоритмы являются истинным источником всей сложности мира

- Детерминированные процессы могут проявлять довольно сложное случайное поведение. Например, генераторы случайных чисел построены как детерминированные процессы
- Вольфрам показал, что простые вычислительные механизмы могут существовать в природе и проявлять все те сложные поведения, которые можно наблюдать в экспериментах. Он приводит множество примеров, начиная от распределения пигментации животных формы и раскраски раковин, развития турбулентности (например, дым в воздухе). Вольфрам делает вывод, что все такие вычисления весьма просты и вездесущи
- В частности, самоорганизующиеся системы (например, нейронные сети, Марковские системы) начинают с простейшей сети, но в процессе функционирования проявляют глубоко интеллектуальное поведение. Это можно видеть на примере мозга человека, который начинает свое развитие с 12 миллионов байтов кодировки в геноме и заканчивает в несколько миллионов раз более сложной структурой, чем начальная

Задача Дж.Майхилла о синхронизации цепи стрелков

Firing squad problem

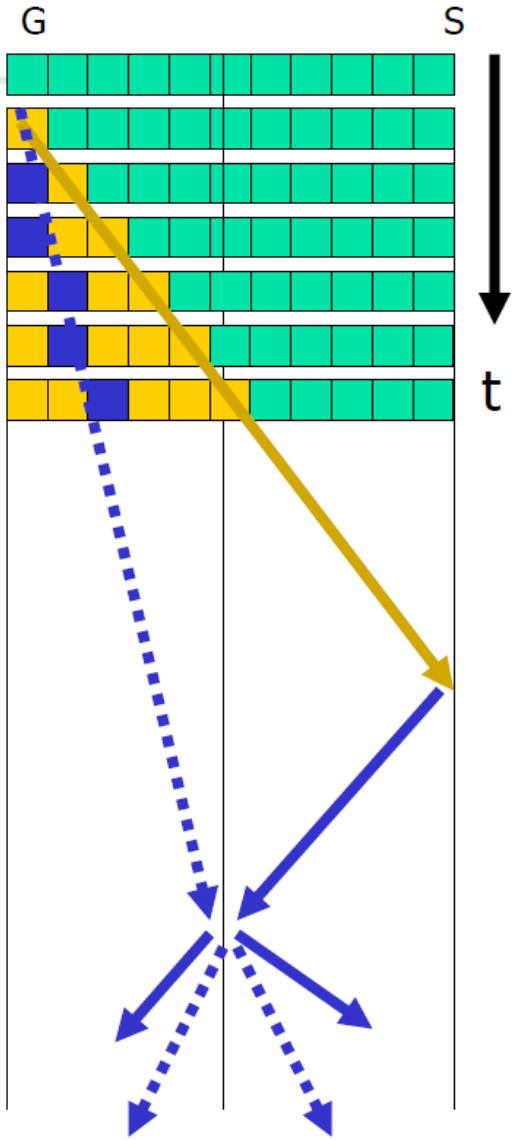
Имеется цепь (линейка) стрелков, с одной стороны которой – генерал, с другой – сержант. Солдаты общаются только со своими соседями, вся система работает по тактам. Солдаты не умеют считать. На каждом такте новое состояние каждого солдата является функцией его старого состояния и пары состояний соседей.

В первом такте генерал изменяет свое состояние от "нейтрального" на "приготовиться"

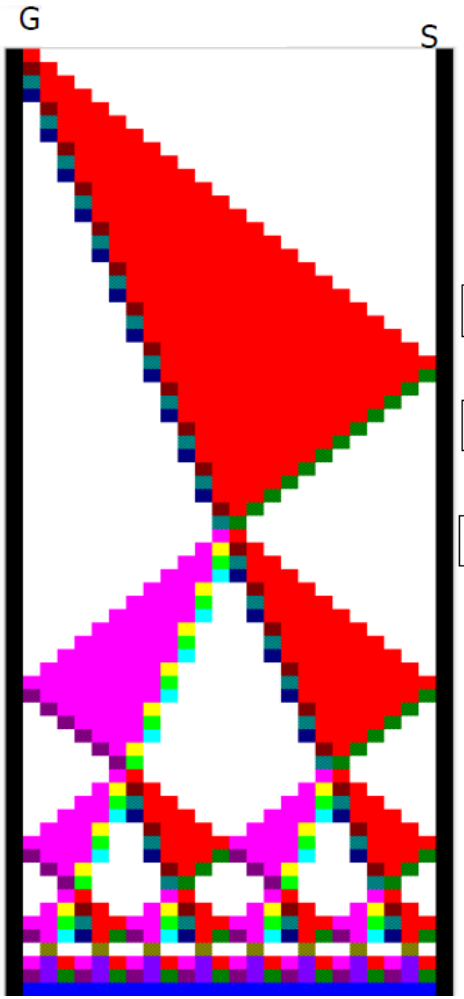
Можно ли построить системы переходов автоматов, моделирующих солдат, так, чтобы все выстрелили в один и тот же момент времени при произвольном числе стрелков?

Задача имеет приложение к делению клеток (биология)

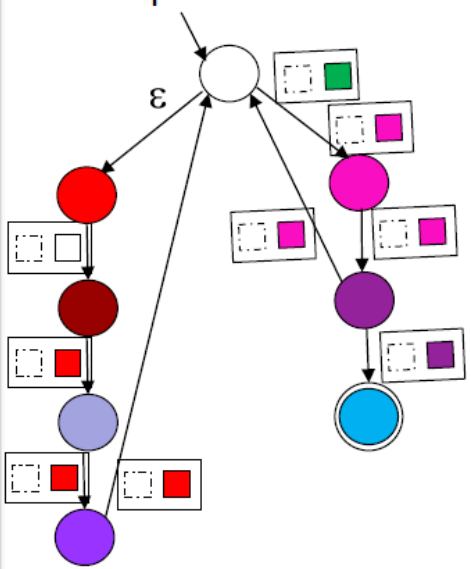
Задача имеет множество решений. Одно из самых простых - 9 состояний у солдата



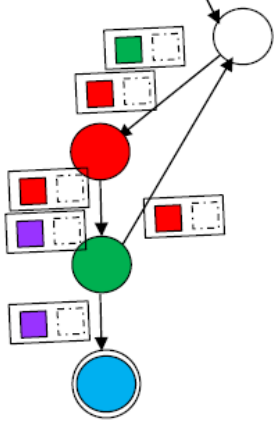
Пример решения: 15 состояний



Генерал:

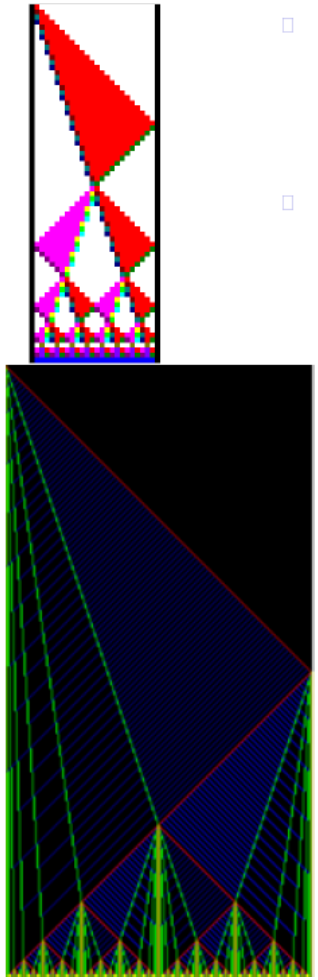


Сержант:



- Проблема: построить автомат солдата (из 15 состояний) и проверить на нескольких линейках солдат.

Примеры решений проблемы синхронизации цепи стрелков



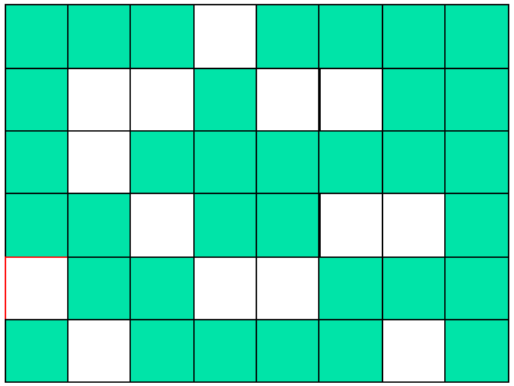
- The name of the problem comes from an analogy with real-world firing squads: the goal is to design a system of rules according to which a general can command a squad of troops to fire, and cause them to all fire their guns simultaneously.
- More formally, the problem concerns cellular automata, arrays of finite state machines called "cells" arranged in a line, such that at each time step each machine transitions to a new state as a function of its previous state and the states of its two neighbors in the line. For the firing squad problem, the line consists of a finite number of cells, and the rule according to which each machine transitions to the next state should be the same for all of the cells interior to the line, but the transition functions of the two endpoints of the line are allowed to differ, as these two cells are each missing a neighbor on one of their two sides.

The states of each cell include three distinguished states: "active", "quiescent", and "firing", and the transition function must be such that a cell that is quiescent and whose neighbors are quiescent remains quiescent. Initially, at time $t = 0$, all states are quiescent except for the cell at the far left (the general), which is active. The goal is to design a set of states and a transition function such that, no matter how long the line of cells is, there exists a time t such that every cell transitions to the firing state at time t, and such that no cell belongs to the firing state prior to time t.

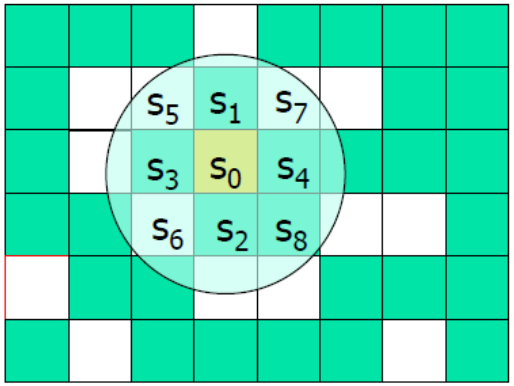
Двумерные клеточные автоматы

Двумерные клеточные автоматы более сложны. При конечном числе состояний можно всю систему представить решеткой с раскрашенными клетками. Если состояний всего два, их можно рассматривать, как “живые” и “мертвые”

шаг k



шаг k+1



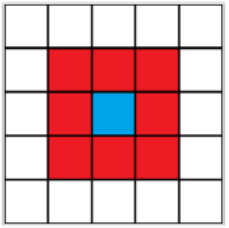
$s'_0 = F_{Мура}(s_0, s_1, s_2, s_3, s_4, s_5, s_6, s_7, s_8)$ либо $s'_0 = F_{Неймана}(s_0, s_1, s_2, s_3, s_4)$

Каждый автомат имеет окрестность
- 8 ближайших соседей - окрестность Мура, либо
- 4 ближайших соседа, 2 по горизонтали и 2 по вертикали – окрестность фон Неймана

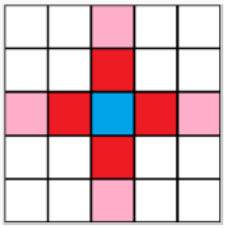
Следующее состояние каждого автомата рассчитывается по одному и тому же правилу (свойство ОДНОРОДНОСТИ системы) и является функцией его текущего состояния и состояний клеток его окрестности (свойство ЛОКАЛЬНОСТИ системы)

Двумерные клеточные автоматы – окрестность Мура

Двумерные клеточные автоматы более сложны. При конечном числе состояний можно всю систему представить решеткой с раскрашенными клетками. Если состояний всего два, их можно рассматривать, как "живые" и "мертвые"

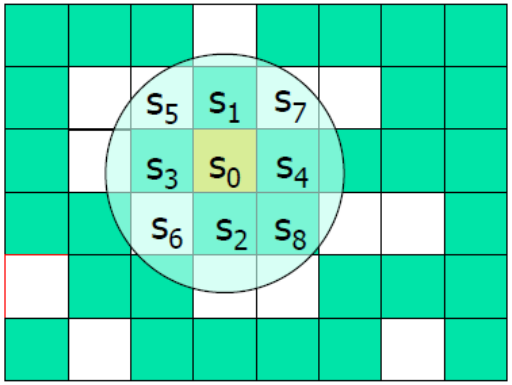


Окрестность Мура



Окрестность фон Неймана

шаг k+1



$2^{2^9} \sim 10^{150}$

возможных функций переходов для автоматов двумерной решетки с двумя состояниями

$s'_0 = F_{Мура}(s_0, s_1, s_2, s_3, s_4, s_5, s_6, s_7, s_8)$ либо $s'_0 = F_{Неймана}(s_0, s_1, s_2, s_3, s_4)$

Если всего два состояния, т.е. только "мертвый-живой" то для окрестности Мура число вариантов такого поведения = числу двоичных функций $2\uparrow(2\uparrow9) = 2\uparrow512 \sim 2\uparrow(10\times50) = 1024\uparrow50 \sim 10^{150}$

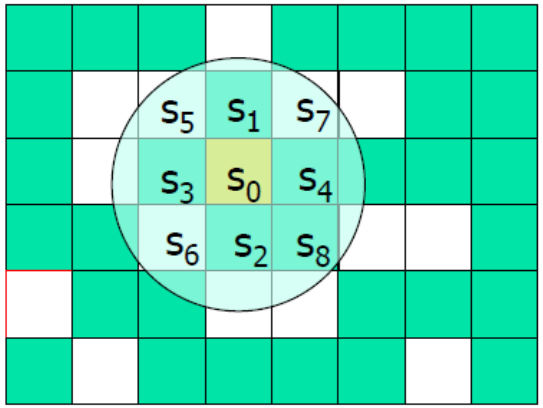
Игра “Жизнь”

Игра “Жизнь” – это клеточный автомат, придуманный английским математиком Джоном Конвеем в 1970 году. Джон Конвей заинтересовался проблемой, предложенной в 1940-х годах известным математиком Джоном фон Нейманом: создать гипотетическую машину, которая может воспроизводить сама себя

Впервые описание игры “Жизнь” было опубликовано в Scientific American №10, 1970, в рубрике «Математические игры» Мартина Гарднера (соседями считаются клетки окрестности Мура, все 8 соседей)

$s'_0 = F(s_0, s_1, s_2, s_3, s_4, s_5, s_6, s_7, s_8)$

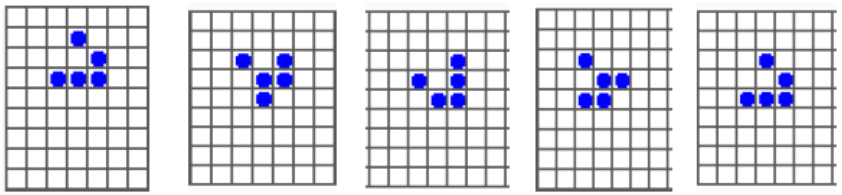
Как удобным способом задать эту функцию от 9 двоичных переменных, чтобы она имела какой-то содержательный смысл?



Двумерные клеточные автоматы. Игра “Жизнь”

- Двумерный клеточный автомат на бесконечном поле (поверхность тора)
- У каждого автомата (клетки) – два состояния: 1- живое и 0 - мёртвое
- Следующее состояние клетки зависит от ее текущего состояния и состояний 8 ее соседей
- Следующие состояния каждой клетки определяется по 3 правилам:
  - мёртвая с тремя живыми соседями оживает;
  - живая:
    - с двумя или тремя живыми соседями продолжает жить;
    - если меньше двух живых соседей, то она умирает (от одиночества)
    - если больше трёх живых соседей, то она умирает (от перенаселенности)

Поведение глайдера:



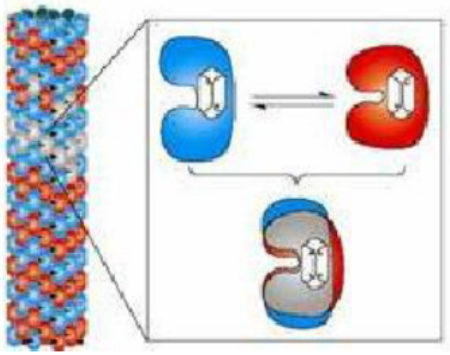
Различных подобных игр можно придумать бесконечное число

Клеточные автоматы

- Придуманная Джоном Конвеем (John Convey) модель “открыла совершенно новую область математических исследований – область клеточных автоматов”
- Простые правила приводят к огромному разнообразию возможной динамики коллектива автоматов
- **Джон Конвей показал, что клеточные автоматы с этими простыми правилами эквивалентны машинам Тьюринга, т.е. с их помощью можно выполнить любой алгоритм**
- Клеточные автоматы являются моделями пространственно распределенных реальных динамических систем
- *Двумерные клеточные автоматы с локальным взаимодействием являются аналогом уравнений в частных производных. Они представляют макроскопические явления множеством точно определенных микроскопических явлений с простейшими законами локальных взаимодействий*.
- В клеточных автоматах совмещены функции хранения и обработки данных (хранение – состояние автомата, обработка – функция переходов в зависимости от данных, хранимых в соседних клетках)

Клеточные автоматы и механизмы сознания

Тубулины



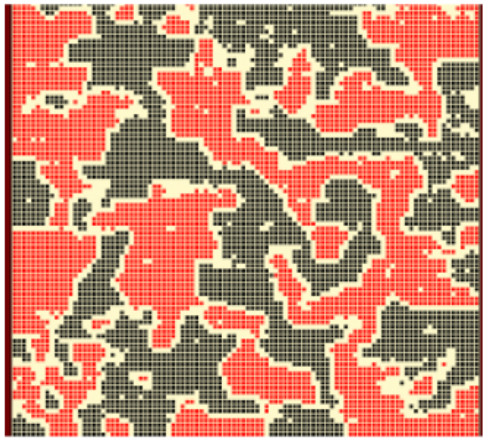
На текущий момент не ясны механизмы обработки информации внутри клетки, не ясны механизмы извлечения и использования результатов

- Каждый нейрон имеет внутреннюю структуру – *цитоскелет* , состоящий из *микротрубочек* . Микротрубочки представляют собой полые трубки, состоящие из белковых молекул – тубулинов. Каждый тубулин содержит около 440 аминокислот.
- Stuart Hameroff установил, что тубулины могут существовать в двух пространственных конфигурациях. Переключение между ними ("операционная редукция") занимает около 10^-9с и осуществляется переходом лишь одного электрона.
- Тубулины в микротрубках образуют большие когерентные квантовые системы, которые могут некоторое время жить "квантовой жизнью", а потом переходить в классическое состояние с помощью "операционной редукции", которая реализуется с помощью белков, сцепляющих тубулины
- Гипотеза С. Хамероффа заключается в следующем.
  1) Цитоскелет нейрона – его вычислительная система
  2) Элементарные операции в ней обеспечиваются квантовыми зацепления состояний соседних тубулинов
  3) Последовательности переключений между двумя конформациями образуют поток сознания
- По сути дела, у нейрона есть "клеточный автомат", изготовленный природой и играющий, по предположению, основную роль в механизме сознания

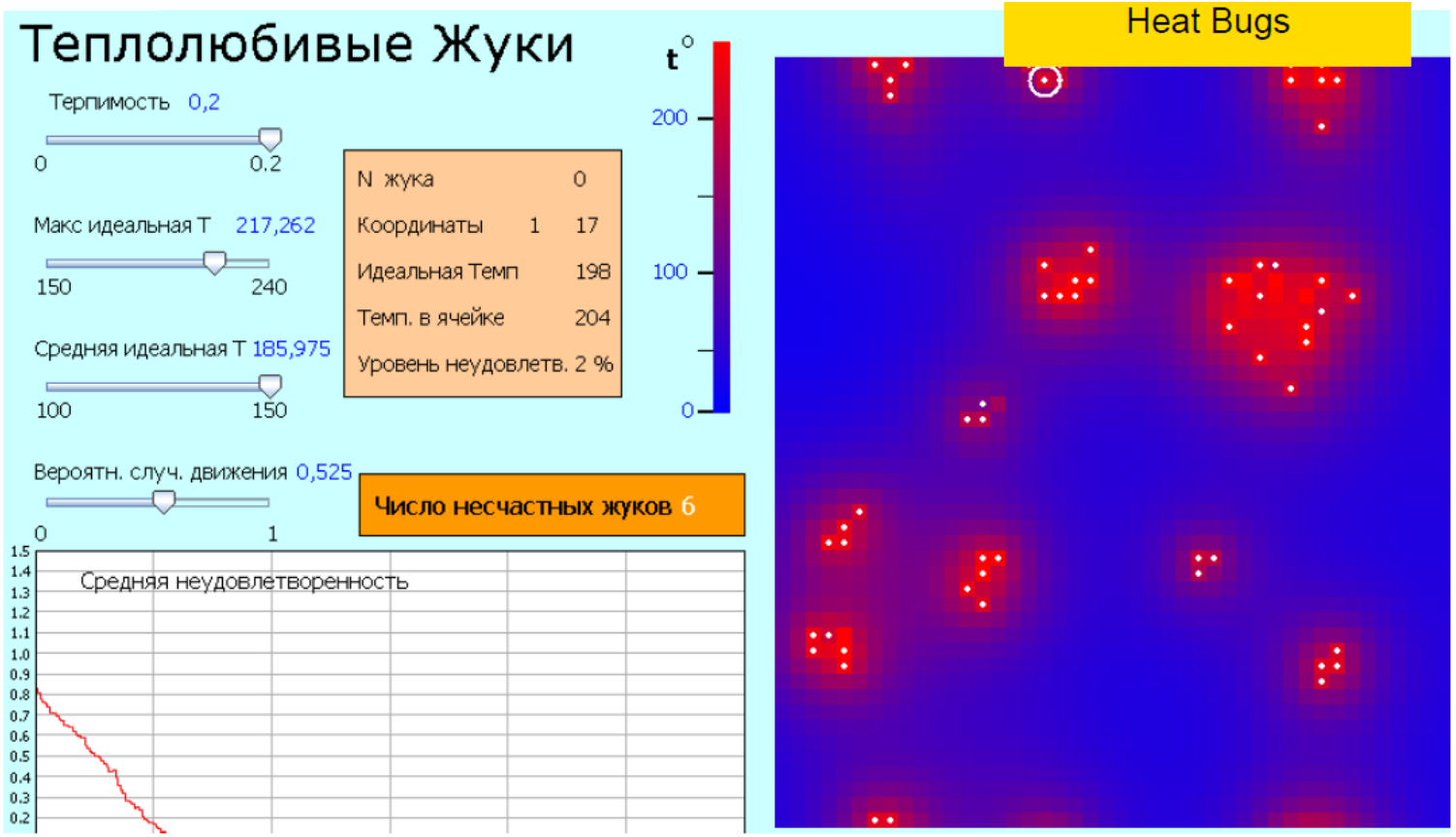
Клеточные автоматы, объясняющие эффект сегрегации

Пример: Thomas Shelling segregation model

Нобелевский лауреат по экономике 2005 г. предложил в 1971 г. простую модель, объясняющую расселение двух групп населения, отличающихся по расовым или национальным признакам



Простая многоагентная модель коллективного поведения

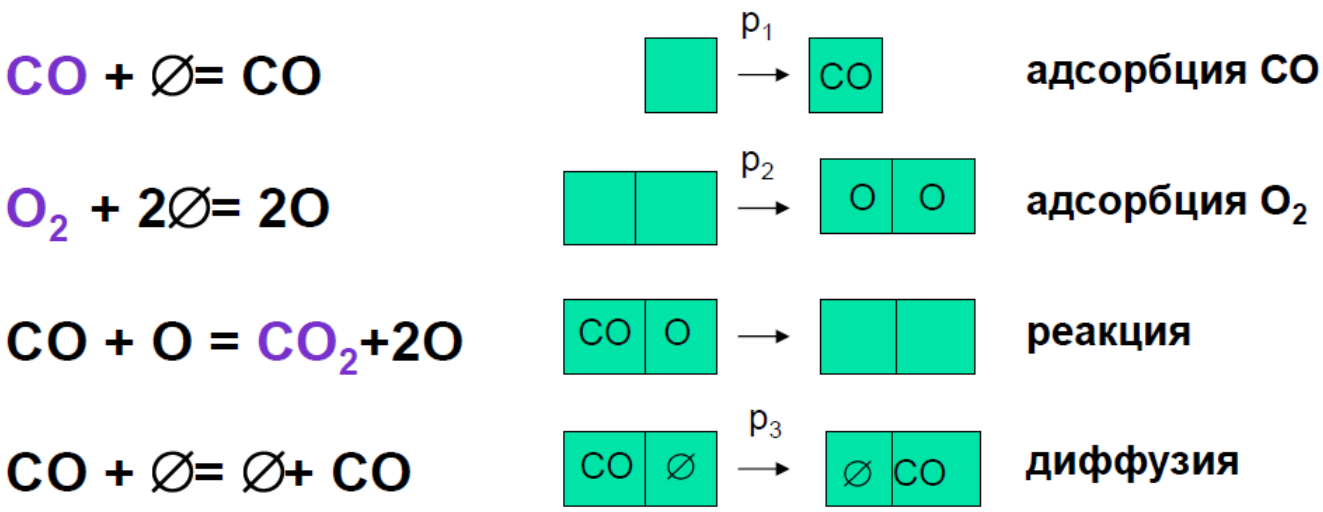


- Моделирование поведения параллельно функционирующих активностей (агентов), взаимодействующих друг с другом и со средой

Моделирование поверхностных химических реакций клеточными автоматами – параллельные вычисления

Пример из работ д.т.н., профессора Ольги Леонидовны Бандман, (Новосибирский Академгородок)

Окисление CO на платиновом катализаторе:



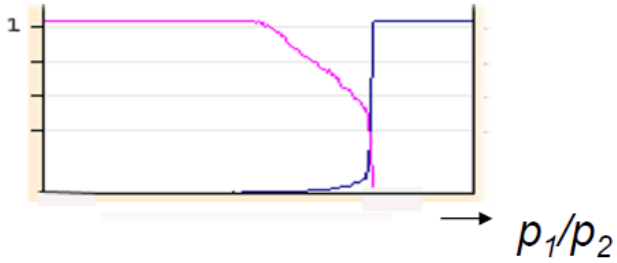
Моделирование поверхностных химических реакций клеточными автоматами



-CO
- O
- Pt

Процесс окисления CO на платиновом катализаторе:

Устойчивые глобальные состояния от отношения парциальных давлений



O_2 (p_2)
CO (p_1)

Моделирование клеточными автоматами процесса самоорганизации

- Формирование устойчивого образа (процесс самоорганизаци)
  A={0,1)
  M={(i,j): I,j=0,...,200}
  Ω(0)={(u, ij): u=1 if *rand* <0.5, u=0 otherwise}

$$u' = \begin{cases} 1, \text{ if } \sum w_k u_k > 0 \\ 0, \ \textit{otherwise} \end{cases}$$

W =

1	0	1	0	0	1	1	
-a	-a	-a	-a	-a	-a	-a	
-a	-a	-a	-a	-a	-a	-a	0
-a	-a	1	1	1	-a	-a	1
-a	-a	1	1	1	-a	-a	0
-a	-a	1	1	1	-a	-a	1
-a	-a	-a	-a	-a	-a	-a	0
-a	-a	-a	-a	-a	-a	-a	0

a=0.2



Заключение

- Конечный автомат – это формальная математическая модель, его можно рассматривать как спецификацию, формальную модель преобразователей информации с конечной памятью, разновидностью формальной модели системы переходов.
- КА можно реализовать – программно и аппаратно, генерируя программу по таблице переходов или автоматически из рисунка – графа переходов
- Как преобразователи информации КА имеют множество применений. Самые простые применения – это построение визуальных алгоритмов обработки событий или текстов с явным выделением состояний. Эта область применения известна с 60-х годов прошлого века
- КА являются естественной моделью визуального задания алгоритмов “реагирующих” систем, в частности, протоколов, драйверов, систем логического управления и т.п.
- Для более компактного и наглядного использования визуального формализма удобно применять стейтчарты (карты состояний), изобретенные в 1987 г. Стейтчарты являются стандартом в UML, при проектировании протоколов и бортовых систем управления. Разработаны генераторы кода - трансляторы из графического языка стейтчартов в ЯВУ
- Расширением КА является модель задания параллельных процессов – сети Петри. КА являются частным классом СП. СП широко используются как формальные модели параллельных процессов.
- Клеточные автоматы – совокупности параллельно работающих взаимодействующих конечных автоматов - являются многообещающей моделью, интенсивное исследование которых продолжается (например, теория Вольфрама, распознавание образов, описание поверхностных реакций, …).



Большое Домашнее задание

Построить программу, позволяющую по заданному номеру БФ визуализировать последовательные состояния линейки клеточных автоматов Вольфрама с различными возможностями:

- Менять начальное состояние линейки
- Предусмотреть источники ошибок при изменении состояний
- Провести анализ нескольких интересных функций



СПАСИБО
ЗА ВНИМАНИЕ